AARHUS UNIVERSITY

Master Thesis

Symbolic Policy Distillation for Interpretable and Trustworthy Deep Reinforcement Learning in Real-World Applications

Students

Anna Sofie Hvid Christensen
Studienr. 202107978

Elisa Lægsgaard
Studienr. 202104652

Sofie Schubert Elving
Studienr. 202105513

Supervisor

Stratis Skoulakis

Co-Supervisor

Alessandro Lucantonio

June 5 2026

30 ECTS
Page count: 56

Aarhus University
Department of Mathematics
Data Science

Contents

Abbreviations

APS Artificial Pancreas Systems.

BB basal-bolus.

BBH basal-bolus human.

BG blood glucose.

CF correction factor.

CFR Critical Failure Rate.

CGM continuous glucose monitoring.

CR carbohydrate to insulin ratio.

DAgger dataset aggregation.

DDPG Deep Deterministic Policy Gradient.

DRL Deep Reinforcement Learning.

FCL fully closed-loop.

GM-DAgger Geometric Mean Dataset Aggregation.

HCL hybrid closed-loop.

IOB Insulin on Board.

PD Proportional-Derivative.

PID Proportional-Integral-Derivative.

PPO Proximal Policy Optimization.

PySR Python Symbolic Regression.

RL Reinforcement Learning.

RPM Revolutions Per Minute.

SAC Soft Actor-Critic.

SP symbolic policy.

SPID Symbolic Policy Interpretable Distillation.

T1DM Type 1 diabetes mellitus.

TAR Time Above Range.

TBR Time Below Range.

TD temporal-difference.

TD3 Twin Delayed Deep Deterministic Policy Gradient.

TIR Time in Range.

TRPO Trust Region Policy Optimization.

1. Abstract

Deep reinforcement learning can learn effective control policies, but neural policies are difficult to interpret and verify, which limits their use in safety-relevant domains. This thesis investigates Symbolic Policy Interpretable Distillation (SPID), a method for distilling trained neural reinforcement-learning policies into compact symbolic controllers.

We implement SPID, reproduce selected benchmark results from the original paper, and apply the method to two control tasks: quadcopter hover control and in-silico blood glucose control for Type 1 diabetes. In both case studies, Proximal Policy Optimization (PPO) is used to train neural teacher policies, which are then distilled into symbolic student policies using symbolic regression.

SPID recovers substantially simpler policies while often retaining useful teacher behavior. In the drone task, the distilled policy resembles a classical feedback controller based on altitude, vertical velocity, and previous motor output. In the diabetes task, several symbolic insulin-dosing policies achieve performance close to their PPO teachers and rely mainly on clinically meaningful variables, including continuous glucose measurements and estimated insulin-on-board.

These results show that SPID can support both interpretable control and post hoc analysis of learned policies. However, symbolic policies remain dependent on teacher quality, reward design, state representation, action mapping, and simulator conditions. SPID therefore appears most useful in structured, low-dimensional simulation settings, while safety-critical deployment requires further robustness testing and domain-specific validation.

2. Introduction

Deep Reinforcement Learning (DRL) has demonstrated strong performance in solving sequential decision-making problems across a wide range of domains. Early landmark results showed that DRL agents could achieve human-level performance in Atari games, while later work demonstrated superhuman performance in Go. Since then, DRL has been successfully applied to a variety of challenging tasks, including robotics, autonomous systems, resource management, recommendation systems, and continuous control problems. Its ability to learn directly from interaction with an environment and to handle high-dimensional state and action spaces has made DRL an attractive framework for developing adaptive decision-making and control strategies in complex settings.

Despite these advantages, the deployment of deep reinforcement learning in safety-critical domains remains challenging. Neural policies are typically regarded as "black-box" models; they are high-dimensional, nonlinear, and difficult to interpret, making it hard to determine why a particular action is chosen. In addition, deep RL methods can be difficult to train reliably and may exhibit limitations related to efficiency, robustness, and generalization. In safety-critical settings, these issues can lead to policies that perform well in simulation but behave unpredictably when exposed to states, patients, or disturbances that differ from those encountered during training. This lack of transparency and predictability is particularly problematic in medicine, where treatment decisions must be clinically justified, inspected for safety, and trusted by practitioners.

To mitigate these challenges, there has been growing attention towards interpretable deep reinforcement learning, with a particular focus on designing policy representations that are easier to inspect, analyze, and verify. Several approaches have been proposed for learning or extracting interpretable policies. One prominent direction represents policies using decision trees. Methods such as VIPER [3], MAVIPER [25], and differentiable decision-tree policies [37] aim to either train decision trees directly or extract them from high-performing neural policies. Decision trees are attractive because their branching structure allows decisions to be traced through a sequence of simple conditions, making them more transparent than deep neural networks. However, this interpretability is often limited by tree size. Shallow trees may be easy to inspect but can suffer from reduced performance, while deeper trees may recover performance at the cost of becoming difficult to understand or verify. Furthermore, decision-tree policies can be poorly suited to smooth continuous control problems, where small changes in the state may lead to abrupt changes in the selected action. Such discontinuities can be undesirable in physical or safety-critical systems, where smooth control signals are often preferred. In these settings, compact algebraic relationships may provide a more natural representation of the policy.

Another line of work has therefore considered programmatic or symbolic policy representations. For instance, Verma et al. proposed PIRL, a programmatically interpretable reinforcement learning framework designed to generate policies that are both interpretable and verifiable [41]. Similarly, Hein et al. proposed GPRL, which uses genetic programming to evolve interpretable algebraic policy equations through model-based reinforcement learning [20]. These methods demonstrate the potential of replacing black-box neural policies with more transparent representations. However, when interpretable policies are learned directly, they often suffer from low data efficiency and reduced performance compared to standard deep RL baselines. This creates a central trade-off between interpretability and control performance.

One approach to mitigating this trade-off is to derive symbolic policies through policy distillation. Policy distillation has been widely used in deep learning to transfer the behavior of a complex teacher model to a simpler student model, but its use for extracting symbolic policies in Reinforcement Learning (RL) remains relatively new. Rather than learning an interpretable policy from scratch, symbolic policy distillation first trains a high-performing DRL policy and then distills its behavior into a symbolic representation. In this setup, the neural policy acts as the teacher, while the symbolic policy serves as an interpretable student. This approach combines two objectives: retaining the performance of the original DRL controller and producing a policy representation that is easier to inspect and analyze. Importantly, the symbolic policy is not only a candidate controller, but also a possible explanation of the teacher policy's behavior, since the resulting expression can reveal which state variables and functional relationships are most important for reproducing the teacher's actions.

In this thesis, we investigate the Symbolic Policy Interpretable Distillation (SPID) framework proposed by Li et al. [22]. SPID distills neural DRL policies into compact analytical expressions using symbolic regression, while balancing fidelity to the teacher with task performance. We implement the SPID framework and first reproduce selected benchmark experiments from the original paper to validate the implementation. We then apply the method to two applied control problems: a quadcopter hover task and an in-silico blood glucose control task for Type 1 diabetes. In both case studies, neural teacher policies are trained and subsequently distilled into symbolic student policies. The resulting policies are evaluated in terms of task performance, fidelity, expression complexity, robustness, and interpretability. The overall aim is to assess whether symbolic policy distillation can retain useful DRL control behavior while producing policies that are simpler, more transparent, and easier to analyze in safety-relevant control settings.

2.1 Reinforcement Learning and Deep Reinforcement Learning

Reinforcement Learning (RL) is concerned with learning from interaction. An agent observes its environment, selects actions, and receives rewards that provide feedback on the consequences of those actions. The aim is to learn a policy that selects actions leading to high expected discounted reward. This interaction is typically formalized as a Markov Decision Process, which provide a mathematical framework for sequential decision marking when outcomes are uncertain.

2.1.1 Markov Decision Processes

A Markov Decision Process (MDP) is commonly defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, P describes the transition dynamics, R is the reward function, and $\gamma \in [0, 1]$ is the discount factor. The Markov property assumes that the distribution of the next state and reward depends only on the current state and action, not on the full history of previous states and actions.

At each time step t, the agent observes the current state s_t, selects an action a_t, and receives a reward (r_{t+1}) while transitioning to a new state (s_{t+1}). The agent's behavior is described by a policy $\pi(a \mid s)$, which defines a probability distribution over actions given the state. The objective is to learn a policy that maximizes the expected discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

where the discount factor, $\gamma \in [0, 1]$, determines the present value of future rewards. Policies are typically assumed to be stationary and Markovian, meaning that the action selection only depends on the current state.[38]

2.1.2 Value Functions and Policy Optimization

In practice, RL methods differ in how they approach the problem of maximizing expected return. Value-based methods estimate *value functions*, which describe the expected future return from a state or state action pair. Policy based methods instead optimize a parameterized *policy* directly. The state value function $v_\pi(s)$ represents the expected return obtained by following a policy π from a given state s, while the action-value function $q_\pi(s, a)$ estimates the expected return obtained by taking action a in state s and subsequently following the policy π [38].

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] && \forall s \in \mathcal{S} \\ Q_\pi(s, a) &= \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a], && \forall s \in \mathcal{S},\ \forall a \in \mathcal{A} \end{aligned}$$

Since value functions estimate the expected return from states or state-action pairs they can be used to derive policies that select high-value actions. Alternatively, policy-based methods optimize a parametrized policy directly.

2.1.3 Deep Reinforcement Learning

In continuous or high-dimensional environments, exact tabular representations of policies and value functions quickly become infeasible. Instead of storing a separate value or action preference for each possible state, Deep Reinforcement Learning (DRL) uses neural networks as function approximators. These networks can approximate policies $\hat{\pi}(a \mid s, \boldsymbol{\theta})$, value functions ($\hat{v}(s, \boldsymbol{\theta})$ or $\hat{q}(s, a, \boldsymbol{\theta})$), or both, where $\boldsymbol{\theta}$ denotes the trainable network parameters [23]. This allows RL methods to scale to complex control problems with continuous observations, nonlinear dynamics, and large state spaces.

This, in combination with RL's ability to learn through interaction with the environment makes DRL particularly useful for the control tasks considered in this thesis, where manually specifying an effective control rule can be difficult.

This flexibility comes at the cost of interpretability. Unlike tabular or simple parametric representations, a deep neural policy encodes its decision rule across many nonlinear transformations, making it difficult to inspect directly. In this thesis, this motivates treating the trained DRL policy as a teacher and distilling its behavior into a simpler symbolic student policy, as described in the following section.

2.2 Symbolic Policy Distillation for Interpretable Reinforcement Learning

Policy distillation is a framework for transferring the behavior of a trained DRL policy, commonly referred to as the teacher policy, to a second and often simpler model called the student policy [32]. The objective is to approximate the decision-making behavior of a complex DRL policy while reducing model complexity or improving interpretability. In practice, the student policy is trained to imitate the outputs of the teacher policy using sampled states from the environment. This is typically formulated as a supervised learning problem, where the student minimizes the difference between the actions predicted by the teacher and the actions produced by the student.

Policy distillation is closely related to imitation learning, in which an agent learns directly from expert demonstrations rather than a reward function. The key distinction is that the teacher in policy distillation is a previously trained RL agent, rather than a human expert. Consequently, the teacher can be queried at any state, which is an important property exploited by the methods described below.

A central challenge in policy distillation is distribution shift, which arises when the student encounters states during execution that differ from those used during training [32]. Since the student is trained on states generated by the teacher's trajectory distribution, any deviation in the student's behavior compounds over time, leading it into regions of the state space that were poorly covered during training. This issue is analogous to the distribution shift problem in imitation learning. A student trained only on teacher-generated trajectories may perform well on the training data, but small deviations can lead it into unseen states where its behavior is poorly constrained. Iterative data collection addresses this by collecting states visited by the student, querying the teacher for the appropriate actions in those states, and retraining the student on the expanded dataset.

In this thesis, the student policy is represented using symbolic regression. Symbolic regression searches for mathematical expressions that fit data while remaining relatively simple and interpretable [33]. Instead of representing the policy through neural network weights, the goal is to obtain an explicit expression mapping state variables to actions. Such symbolic policies can make it easier to inspect which variables influence the controller and how actions are computed.

However, symbolic regression is computationally challenging because the space of possible expressions grows rapidly with the number of variables, operators, and expression length [42]. Practical symbolic regression methods therefore balance predictive accuracy against expression complexity. This trade-off is central to symbolic policy distillation: the student should be simple enough to interpret, while still retaining the useful behavior of the neural teacher.

Symbolic policy distillation therefore provides a bridge between the flexibility of DRL and the inspectability of analytical control rules. In the following methods section, we describe how this idea is implemented using SPID, where iterative dataset aggregation (DAgger) is combined with symbolic regression to distill neural teacher policies into compact symbolic expressions.

3. Data and Methods

The empirical work in this thesis is based on our own implementation of the SPID framework. To validate the implementation before applying it to the drone and glucose-control case studies, we reproduced selected benchmark experiments from the original SPID paper. The following sections introduce the teacher algorithms, the symbolic distillation procedure, and the symbolic regression method needed to understand this implementation and the subsequent experiments.

3.1 Deep Reinforcement Learning Algorithms

The SPID framework distills a trained DRL policy into a symbolic student policy. The trained DRL policy therefore acts as the teacher, providing both action labels and, when available, value information used during distillation. In the benchmark reproduction, we consider several teacher algorithms, including Proximal Policy Optimization (PPO), Trust Region Policy Optimization (TRPO), Deep Deterministic Policy Gradient (DDPG), Twin Delayed Deep Deterministic Policy Gradient (TD3), and Soft Actor-Critic (SAC). These algorithms differ in how they optimize policies, how they collect training data, and whether they maintain an explicit state action-value function. The later case studies focus on PPO, while the remaining algorithms are included primarily to reproduce the original SPID benchmark setting and test the implementation across different classes of teacher policies.

Across benchmarks and case studies, we make use of *Stable Baselines3*, an open-source library for reinforcement learning that provides a suite of reliable, well-tested algorithms in PyTorch, including PPO, SAC, DDPG and TD3. For TRPO we make use of the experimental implementation offered by the Contrib package for Stable Baselines3.

3.1.1 Actor Critic and Policy Gradient Methods

DRL algorithms can be broadly distinguished by whether they primarily estimate value functions, directly optimize policies, or combine both. Value-based methods focus on estimating value functions in order to **evaluate** the long-term quality of states or actions, whereas policy-based methods directly optimize parameterized policies for **selecting** actions [38].

Actor-critic methods combine policy optimization with value-function estimation. The actor represents the policy used to select actions, while the critic estimates value-based quantities used to guide updates of the actor [38, 23].

Depending on the specific algorithm, the critic may estimate either a state-value function, an action-value function, or an advantage function $A(s, a)$. The advantage function, commonly formulated as

$$A(s, a) = Q(s, a) - V(s), \tag{1}$$

measures the performance difference for a selected action compared to the expected value of the current state [23, 22].

The critic is commonly trained using temporal-difference (TD) learning, where value estimates are updated from observed rewards and bootstrapped estimates of future value. For a state-value critic, the one-step TD error is

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

[38]. The TD-error measures the discrepancy between the current value estimate and a one-step bootstrapped target, and therefore provides a learning signal for updating the critic. In actor-critic methods, this signal can also be used to construct an advantage estimate for improving

the actor.

The actor is optimized using policy-gradient methods, which update the parameters of the policy directly to increase the expected return. Rather than deriving actions from a learned value function, policy-gradient methods adjust the policy so that actions associated with higher-than-expected returns become more likely, while actions associated with lower-than-expected returns become less likely.

A key challenge is that overly large updates to the parameters of the policy can make the new policy differ substantially from the policy that generated the training data, leading to unstable learning or sudden performance drops. This motivates the trust-region constraint in TRPO and the clipped objective used in PPO.

3.1.2 On-Policy and Off-Policy Teacher Algorithms

Teacher algorithms also differ in how training data is collected and reused. On-policy methods update the policy using trajectories generated by the current policy. This keeps the data consistent with the policy being optimized, but limits data reuse because trajectories become outdated as the policy changes. PPO and TRPO belong to this family.

Off-policy methods can learn from trajectories generated by a different behavior policy, commonly by storing previous transitions in a replay buffer. This improves sample efficiency, but introduces a mismatch between the data-generating policy and the policy being optimized. DDPG, TD3, and SAC are off-policy actor-critic methods.

3.1.3 Algorithms Used in Benchmark Reproduction

3.1.3.1 Proximal Policy Optimization (PPO) [36] is as mentioned an on-policy policy gradient method and the primary teacher algorithm used in the applied case studies. PPO is currently considered a state-of-the-art methods, and has been successfully applied to various areas such as gaming and robotic control. The algorithm builds on the trust-region idea introduced by TRPO [34], where the aim is to prevent policy updates from changing the policy too abruptly. Large updates can make the new policy differ substantially from the policy that generated the training data, which may lead to unstable learning. PPO addresses this by replacing the hard trust-region constraint used in TRPO with a clipped surrogate objective that can be optimized using standard first-order methods.

$$\mathbb{E}\left[\min\left(\rho_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right].$$

where $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between the new and old policy, and $\hat{A}_t$ is an estimated advantage. The clip function restricts $\rho_t(\theta)$ to the interval $[1-\epsilon, 1+\epsilon]$, limiting the contribution of updates that move the new policy too far from the old policy. When the advantage $\hat{A} > 0$, the update increases the probability of the action but only up to a factor of $1+\epsilon$. When $\hat{A} < 0$, it decreases it but only down to $1-\epsilon$.

PPO follows the actor-critic method described above, using π_θ as the actor and $V_\phi(s)$ as the critic. The critic is used to compute advantage estimates via Generalized Advantage Estimation (GAE) [35]:

$$\hat{A}_t = \sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}, \quad \text{where} \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

where λ controls the bias-variance trade-off. A typical PPO training loop alternates between collecting trajectories under the current policy, estimating returns and advantages, and performing several epochs of minibatch gradient updates

3.1.3.2 Off-policy actor-critic algorithms The remaining benchmark algorithms, DDPG [24], TD3 [16], and SAC [18], are off-policy actor-critic methods for continuous-action control. They are included in the benchmark reproduction because they represent commonly used continuous-control algorithms and, unlike PPO, maintain explicit action-value functions.

DDPG learns a deterministic policy together with a critic Q(s,a). The policy is updated to choose actions with high estimated Q-values, but this can make the method sensitive to overestimation errors in the critic. TD3 was introduced to reduce this instability by using two critics, delayed policy updates, and smoothing of the target actions.

SAC also learns explicit Q-functions, but uses a stochastic policy and an entropy-regularized objective. This encourages the policy to achieve high reward while maintaining exploration. Like TD3, SAC uses two critics to reduce overestimation bias, while a temperature parameter controls the relative weight of the entropy term.

3.2 Symbolic Policy Distillation

After training a DRL teacher policy, the next step is to distill its behavior into a symbolic student policy. A simple approach would be to collect trajectories from the teacher and train the student to imitate the teacher's actions on this fixed dataset. However, this can lead to distribution shift: once deployed, the student may make small errors that take it into states not well represented in the teacher-generated data. The distillation method used in this thesis is therefore based on SPID, which combines iterative dataset aggregation with symbolic regression. This section describes how training data are collected, how student policies are evaluated against the teacher, and how symbolic regression is used to obtain compact analytical policies [22].

3.2.1 Dataset Aggregation

dataset aggregation (DAgger), is an iterative imitation learning algorithm designed to address distribution shift [31]. Rather than training on a fixed set of expert demonstrations, DAgger iteratively collects new data from the states the student itself visits, queries the teacher for corresponding actions, and retrains on the aggregated dataset. This ensures the training distribution converges toward the student's own state visitation distribution. The used training loss is often based how well the student policy mimics the teacher, regardless of the state and it's importance.

Q-DAgger extends the DAgger loss by weighting errors according to their long-term performance impact [3]. Rather than penalizing all deviations equally, it leverages the teacher's Q-function to measure how much value is lost when the student deviates from the teacher's optimal action. However, optimizing only on this performance objective can result in symbolic student policies that deviate significantly from the teacher's actual behavior. This can in some cases enable the student policy to perform better than the teacher, but can also lead to unwanted deviations if the goal is to faithfully represent the teacher's decision making.

3.2.2 GM-DAgger

The central part of SPID is Geometric Mean Dataset Aggregation (GM-DAgger), which combines the objectives of DAgger and Q-DAgger into a single balanced loss [22]. As described above, DAgger ensures the distilled policy stays close to the teacher's behavior but ignores whether the imitated actions are actually valuable. Q-DAgger instead focuses on maintaining performance, but a policy that achieves high reward may still behave in ways that are inconsistent with the teacher. For interpretable RL, both properties matter: fidelity enables the symbolic policy to serve as a transparent surrogate for the teacher, while performance ensures it remains a useful agent.

Hence, GM-DAgger combines two complementary objectives; a performance gap and a fidelity gap. The performance gap corresponds to the Q-DAgger loss and measures how much value is lost relative to the teacher when a student takes action $\pi(s)$ in state s at time t:

$$g_t^p(s,\pi) = V_t^{\pi^*}(s) - Q_t^{\pi^*}(s,\pi(s)) + \alpha \tag{2}$$

where π^* is the teacher policy, π is the student policy and $\alpha > 0$ ensures positivity. Recalling the advantage function defined in equation ((1)), this term can equivalently be written as $-A^{\pi^*}(s,\pi(s)) + \alpha$, meaning that the performance gap penalizes actions with negative advantage under the teacher's value estimates. This notation is what is used in the algorithm, See Algorithm 1 more specifically operation 28.

The fidelity gap corresponds to the DAgger loss and measures the divergence between the student policy and the teacher policy:

$$g_t^f(s,\pi) = \|\pi(s) - \pi^*(s)\|^2 + \epsilon \tag{3}$$

where $\epsilon > 0$ ensures positivity.

The GM-DAgger loss combines these through their geometric mean to form the loss:

$$\ell(s,\pi) = \sqrt{g^p(s,\pi) \cdot g^f(s,\pi)}$$

3.2.2.1 SPID Training Procedure [22] The neural network teacher policy π^* is trained using any standard DRL algorithm. A dataset is then created by collecting state-action pairs from the teacher and fitting an initial symbolic policy $\hat{\pi}_0$ through symbolic regression. This initial policy tends to performs poorly due to distribution shifts, as it tends to follow trajectories that are unseen in the teacher's trajectories. As a result, SPID applies data aggregation similar to DAgger. At each iteration, the trajectories are collected from a mixture of the teacher policy and the symbolic policy using a mixing coefficient β:

$$\pi_i = \beta\pi^* + (1-\beta)\hat{\pi}_i$$

where the default value for $\beta = 0.5$. Hence, the new data is aggregated with the existing dataset and the student is retrained using the GM-DAgger loss via symbolic regression. This is repeated for a fixed number of iterations, and the best performing policy across all iterations is then returned. The full procedure is summarized in Algorithm 1.

The symbolic learner used in SPID is Python Symbolic Regression (PySR) [10], a high performance symbolic regression library based on a multi-population evolutionary algorithm, as described in Section 3.2.3. At each iteration, the symbolic student is retrained on the aggregated dataset, which ensures the student progressively adapts to the states it actually encounters during execution, rather than only those visited by the teacher.

3.2.2.2 Estimating Q-values Computing the performance gap in equation (2) requires access to the teacher's Q-function $Q^{\pi^*}(s,a)$. In most cases, these Q-values are not directly available, as they depend on the specific training algorithm used for the teacher.

For off-policy actor-critic algorithms such as SAC, TD3, and DDPG, an explicit Q-network is maintained as part of the training procedure and can be queried directly. For on-policy algorithms such as PPO, and TRPO, the critic estimates the state-value function $V^{\pi^*}(s)$ instead of the full Q-function. In this case, Q-values are estimated using a one-step TD bootstrap[1]:

$$Q^{\pi^*}(s,a) \approx r + \gamma \cdot V^{\pi^*}(s')$$

[1]This approximation is an implementation detail not stated explicitly in the original SPID paper. We thank the authors of SPID [22] for clarifying the value-estimation procedure used for on-policy teacher algorithms.

where s' is the next state following action a in state s, and r is the observed reward. We insert this in equation (2) an the performance gap then becomes:

$$g_t^p(s,\pi) \approx V_t^{\pi^*}(s) - (r + \gamma \cdot V^{\pi^*}(s')) + \alpha$$

Algorithm 1 Symbolic Policy Interpretable Distillation (SPID)

Input: MDP $(\mathcal{S}, \mathcal{A}, P, R)$; teacher policy π^*; teacher value estimator V^* or Q^*; discount factor γ; positivity constants α, ϵ; mixing coefficient β; trajectories per iteration M; number of iterations N
Output: Best symbolic policy $\hat{\pi}$

Initialize dataset $\mathcal{D} \leftarrow \varnothing$
Initialize symbolic policy $\hat{\pi}_0$ from teacher rollouts

for $i = 0$ to N **do**

  // Collect trajectories from mixture policy
  Set mixture policy $\pi_i = \beta\pi^* + (1-\beta)\hat{\pi}_i$
  Collect M trajectories using π_i
  Store transitions $\mathcal{D}_i = (s, a, r, s', \pi^*(s))$

  for all $(s, a, r, s', \pi^*(s)) \in \mathcal{D}_i$ **do**

    // Estimate performance signal from available teacher value information
    if teacher has state-value critic V^* **then**
      $\widehat{Q}^{\pi^*}(s,a) \leftarrow r + \gamma V^*(s')$
      $\widehat{z}^p(s,a) \leftarrow \widehat{Q}^{\pi^*}(s,a) - V^*(s)$
    else if teacher has action-value critic Q^* **then**
      $a' \leftarrow \pi^*(s')$
      $\widehat{z}^p(s,a) \leftarrow Q^*(s,a) - (r + \gamma Q^*(s', a'))$
    else
      $\widehat{z}^p(s,a) \leftarrow r - \bar{r}$
    end if

    // Compute GM-DAgger loss components
    $g^p(s,a) \leftarrow \alpha - \widehat{z}^p(s,a)$
    $g^f(s,\hat{\pi}) \leftarrow |\hat{\pi}(s) - \pi^*(s)|^2 + \epsilon$
    $\ell(s,\hat{\pi}) \leftarrow \sqrt{g^p(s,a) \cdot g^f(s,\hat{\pi})}$
  end for

  // Aggregate data and retrain symbolic policy
  $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D} * i$
  $\hat{\pi} * i + 1 \leftarrow$ **SymbolicRegression**$(\mathcal{D}, \ell)$
end for

return Best policy $\hat{\pi} \in \hat{\pi} * 1, \ldots, \hat{\pi} * N + 1$ on validation

3.2.3 Symbolic Regression with PySR

The symbolic learner used in SPID is PySR [10], a symbolic regression library with a Python interface to *SymbolicRegression.jl*. PySR searches for analytical expressions that map input variables to target outputs while balancing predictive accuracy and expression complexity. In symbolic policy distillation, the inputs correspond to the policy state representation, while the

targets are the teacher actions.

PySR uses a multi-population evolutionary search over candidate expressions represented as trees, where leaves are variables or constants, and internal nodes are mathematical operators. Candidate expressions are modified through mutation, crossover, simplification, and optimization of numerical constants. [10].

The PySR search space is controlled by the allowed operators and by constraints on expression size, depth, and nesting. The operator set determines which functional forms can appear in the policy. Simple arithmetic operators tend to produce smooth and interpretable expressions, while division, logarithms, exponentials, trigonometric functions, and threshold operators increase expressivity but may also reduce interpretability or introduce numerical instability.

Expression complexity is defined in terms of the number of nodes in the expression tree, including variables, constants, and operators. PySR can discourage unnecessarily complex expressions through a parsimony penalty, which adds a complexity-dependent term to the loss. In addition, maximum depth and nesting constraints restrict how deeply operations can be composed, thereby limiting overly nested expressions that may be difficult to interpret or unstable outside the sampled state distribution [10].

These choices are treated as methodological parameters in this thesis. A more expressive PySR configuration may improve fidelity to the teacher, but can also produce longer and less robust expressions. A restricted search space encourages compact policies, but may limit the student's ability to reproduce the teacher behavior. In the case studies, operator sets, expression size, depth, and nesting constraints are therefore varied or restricted to study their effect on performance, fidelity, numerical stability, and interpretability.

3.3 Benchmark Environments

OpenAI offers an open-source Python toolkit and API used to develop, test and benchmark RL algorithms, *Gymnasium* [40]. Gymnasium offers different simulated environments, ranging from classic control problems to video games. The SPID-algorithm is benchmarked on six classical control environments; *Cartpole*, *MountainCar*, *Pendulum*, *Swimmer* and *Reacher* using the five different DRL-models; PPO, TRPO, DDPG, SAC and TD3. To verify and test our implementation of the SPID-algorithm, we recreate the results of the paper [22], training a model for each combination DRL-algorithm and environment. A short description of the environments and the tasks are offered in the sections below, and final results can be found in Section 3.3.6.

3.3.1 Cartpole

Cart pole as described in [2]. Actions consist of pushing the cart either left or right, given a state described by the cart position, velocity, the angle of the pole and the angular velocity.

The goal is to keep the angle upright for as long as possible, with a reward of +1 for every step taken (including termination step) and a reward threshold of 500 due to time limit.

CartPole is originally defined with a discrete action space consisting of two actions corresponding to applying a fixed force to the left or right. Since DDPG is designed for continuous control tasks, we follow common practice in the literature and convert CartPole into a continuous-action environment. Specifically, the agent outputs a one-dimensional continuous action, which is mapped to the original discrete actions by thresholding: positive actions apply a force to the right, while non-positive actions apply a force to the left. The environment dynamics and reward structure remain unchanged.

3.3.2 MountainCar

MountainCar models a car positioned in a valley between two hills. The engine of the car is not powerful enough to drive directly up the slope to the goal. The objective is therefore to apply force to the car in order to build momentum by repeatedly driving back and forth between the hills until the car reaches the top of the right hill.

The action space is continuous and represents the magnitude of the force applied to the car's engine. Positive values accelerate the car forward, while negative values accelerate it in the opposite direction. The state space typically consists of the car's position and velocity. The reward structure penalizes each timestep with a reward of -1 until the car reaches the goal position at the top of the hill, at which point the episode terminates and gets a reward of $+100$ added to the negative reward for that timestep.

The task is particularly challenging because the agent can only gain momentum by moving away from the target. This leads to an optimal policy that must account for long-term consequences, requiring planning over multiple time steps.

3.3.3 Pendulum

The Pendulum environment models a single-link pendulum that is free to rotate around a fixed pivot. The goal is to apply torque in order to swing the pendulum up and keep it balanced in the upright position.

The action space is continuous and consists of a single value representing the torque applied at the joint. The state space is typically defined by the cosine and sine of the pendulum angle, as well as its angular velocity. Representing the angle using sine and cosine avoids discontinuities at angle wrap-around.

The reward function penalizes deviations from the upright position as well as large angular velocities and control effort. It is commonly defined as:

$$r = -(\theta^2 + 0.1\dot{\theta}^2 + 0.001u^2), \tag{4}$$

where θ is the angle from the upright position, $\dot{\theta}$ is the angular velocity, and u is the applied torque.

Unlike environments such as CartPole, the Pendulum task does not terminate early upon failure. Instead, episodes run for a fixed number of 200 timesteps. This results in a dense reward signal, making the environment suitable for evaluating continuous control algorithms and their ability to learn smooth control policies.

3.3.4 Swimmer

The Swimmer environment models a snake-like robotic agent consisting of several connected links moving in a two-dimensional pool environment. The objective is to learn coordinated movements that move the agent forward as efficiently as possible.

The agent is controlled through a continuous action space consisting of torques applied to the swimmers' joints. In the standard Swimmer-v5 environment, two joints are activated, resulting in a two-dimensional action space. The state space contains joint configurations, angular velocities, and velocity of the swimmers body.

The reward function encourages the swimmer to move forward while discouraging unnecessarily large actions:

$$r = r_{\text{forward}} - \lambda \|a\|_2^2$$

where r_{forward} is the forward velocity reward, a is the action and λ is the penalizing weight. The environment therefore rewards efficient locomotion behavior rather than simply maximizing speed.

Each episode runs for 1000 timesteps during which the agent continuously interacts with the environment in order to maintain forward movement. Since the swimmer must coordinate multiple connected body segments, the environment introduces more complex dynamics than single-body control tasks. Swimmer is commonly used as a benchmark environment and is particularly useful for evaluating how well algorithms learn coordinated motion and stable control strategies in systems with continuous state and action spaces.

3.3.5 Reacher

The Reacher environment models a two-link robotic arm operating in a two-dimensional plane. The objective is to control the joints of the arm such that the end-effector reaches a randomly positioned target.

The action space is continuous and consists of torques applied to each of the two joints, meaning that the action space is 2-dimensional just like Swimmer. The state space includes joint angles, angular velocities, and the position of the target relative to the end-effector.

The reward function is composed of two parts: a distance-based penalty and a control penalty. It is commonly defined as:

$$r = -\|x_{\text{end}} - x_{\text{target}}\|^2 - \lambda \|u\|^2, \tag{5}$$

where x_{end} is the position of the end-effector, x_{target} is the target position, u is the applied torque, and λ is a small coefficient penalizing large actions.

Episodes run for a fixed number of 50 timesteps, and the target position is randomly sampled at the beginning of each episode. This introduces variability in the task and requires the agent to generalize across different target locations.

Reacher is typically used a benchmark for continuous control and robotic manipulation. Compared to simpler environments such as Pendulum, it introduces higher-dimensional state and action spaces, as well as more complex dynamics, making it a useful testbed for evaluating the performance of modern RL algorithms.

3.3.6 Results

Due to time restriction, not all benchmarks were trained from scratch. For troublesome models, pretrained models were used as teachers, and SPID was applied to extract symbolic student

policies from these models, since validating the distillation pipeline was the primary objective of this section.

Table 1 compares the evaluation performance of the neural teacher policies with the corresponding SPID-distilled symbolic policies. The results show that the symbolic policies often retain performance close to the teacher, particularly in simpler environments such as CartPole. In some cases, the symbolic policy matches or exceeds the reported teacher performance, while in more complex environments such as Swimmer and Reacher, the performance gap is more variable. This is expected, since higher-dimensional continuous-control tasks require richer policy representations and are therefore more challenging to express with compact symbolic formulas.

Env	Deep RL algo	Performance	SPID distillation
CartPole	PPO	482.6 $\pm$27.6	500.0 $\pm$0.0
	TRPO	500.0 $\pm$0.0	500.0 $\pm$0.0
	DDPG	500.0 $\pm$0.0	500.0 $\pm$0.0
	SAC	500.0 $\pm$0.0	500.0 $\pm$0.0
	TD3	500.0 $\pm$0.0	500.0 $\pm$0.0
MountCar	PPO	94.0 $\pm$1.2	89.5 $\pm$3.1
	TRPO	92.5 $\pm$0.3	93.6 $\pm$0.7
	DDPG	94.5 $\pm$0.0	95.4 $\pm$9.8
	SAC	94.7 $\pm$1.3	90.7 $\pm$4.9
	TD3	93.5 $\pm$0.1	94.2 $\pm$0.6
Swimmer	PPO	297.0 $\pm$1.8	232.9 $\pm$1.5
	TRPO		
	DDPG		
	SAC	316.1 $\pm$1.8	314.2 $\pm$3.2
	TD3	311.9 $\pm$3.1	333.5 $\pm$6.0
Pendulum	PPO	-161.6 $\pm$99.2	-168.9 $\pm$103.0
	TRPO	-191.2 $\pm$172.5	-214.5 $\pm$128.7
	DDPG	-146.2 $\pm$96.2	-169.9 $\pm$79.5
	SAC	-131.1 $\pm$76.5	-188.6 $\pm$123.0
	TD3	-148.6 $\pm$82.2	-147.2 $\pm$91.5
Reacher	PPO	-4.4 $\pm$2.2	-7.9 $\pm$4.5
	TRPO	-10.6 $\pm$4.5	-10.5 $\pm$3.5
	DDPG	-3.4 $\pm$1.5	-8.5 $\pm$2.3
	SAC	-3.2 $\pm$1.3	-9.6 $\pm$4.6
	TD3	-3.9 $\pm$1.4	-8.2 $\pm$3.2

Table 1: Overview of selected DRL algorithm performance on classical control problems, compared to the performance of the SPID-extracted symbolic policies. Empty entries for Swimmer with TRPO and DDPG indicate that no suitable pretrained model configurations could be identified for these combinations.

Overall, the benchmark reproduction supports that the implementation captures the main intended behavior of SPID. The symbolic policies are not guaranteed to match the teacher in every environment, but they frequently recover useful control behavior with substantially simpler representations. This provides confidence that the implementation is suitable for the subsequent case studies, where the focus shifts from reproducing benchmark results to studying symbolic distillation in more application-specific control settings.

During these experiments, we observed that strong symbolic policies could often be obtained from relatively small amounts of rollout data. In several cases, short iterative data-aggregation runs produced better symbolic students than longer rollouts. This may indicate that repeated aggregation of diverse student-visited states is more important than simply collecting large numbers of samples from a few trajectories. However, this should be interpreted cautiously.

Since the GM-DAgger performance term depends on rewards and value estimates, very short or incomplete trajectories may provide a weaker signal in environments with delayed or sparse rewards. The benchmark results should therefore be understood as an implementation check rather than a full analysis of optimal SPID training settings.

Having confirmed that the implementation reproduces the core behavior of SPID across a range of standard environments and teacher algorithms, we proceed to apply the framework to two applied control problems in Sections 4 and 5, where symbolic distillation is studied in more realistic and safety-relevant settings.

4. Case Study: Flying a Drone Using a Symbolic Policy

4.1 Introduction

Quadcopter control is a well-studied problem in robotics and autonomous systems, involving nonlinear dynamics, continuous control inputs, and strong coupling between motor commands and vehicle motion. This makes it a relevant setting for evaluating RL-based controllers. At the same time, autonomous aerial vehicles are increasingly considered for applications where controller behavior should be robust, inspectable, and compatible with safety constraints [44]. These properties make drone control a useful case study for symbolic policy distillation.

For this purpose, we use the gym-pybullet-drones environment [28], an open-source Gymnasium-compatible simulation of quadcopter control based on the Bullet physics engine [9], compatible with the Stable Baselines3 workflow. We consider a simplified single-drone hover task with symmetric motor control, as described in Section 4.2. This setup is intentionally kept simple to provide a clean and interpretable testbed for evaluating symbolic policy distillation outside the standard benchmark environments. The low-dimensional and structured nature of the task makes it well suited for initial exploration of SPID hyperparameters and for building intuition about how the distillation procedure behaves in a physics-based environment, before considering the more complex and safety-critical blood glucose control case study in Section 5.

4.2 Drone Environment

The environment supports both single- and multi-agent simulation. At each timestep, the environment returns an observation vector for each simulated drone. For a single drone, the observation vector is

$$[\mathbf{x}, r, p, j, \dot{\mathbf{x}}, \boldsymbol{\omega}, [P_0, P_1, P_2, P_3]] \tag{6}$$

where $\mathbf{x} = [x, y, z]$ contains the three-dimensional position, r, p and j is the roll, pitch and yaw angles, respectively, $\dot{\mathbf{x}}$ are the linear velocities, $\boldsymbol{\omega}$ are the angular velocities, and $[P_0, P_1, P_2, P_3]$ is the corresponding four motor speeds.
In addition to the base observations, the environment also maintains an action buffer that stores the previous actions taken within the past 0.5 seconds, corresponding to approximately the 15 most recent actions taken. The full observation vector therefore concatenates Equation (6) with the action buffer, resulting in a 27-dimensional observation space. In the multi-agent case, such a vector is received independently for each drone.

Advancing the environment by one step requires passing an action defining the control input for each drone. Two control modes are available. The first is to directly specify the four motor speeds in Revolutions Per Minute (RPM) for each drone n:

$$\{\, n : [P_0, P_1, P_2, P_3]_n \,\} \tag{7}$$

The second option is to pass a desired velocity vector:

$$\{\, n : [v_x, v_y, v_z, v_M]_n \,\} \tag{8}$$

where v_x, v_y, v_z are the components of a unit direction vector and v_M is the desired velocity magnitude. In this mode, the translation from desired velocity to motor speeds is handled internally by a low-level Proportional-Integral-Derivative (PID) controller embedded within the environment [28]. Reward functions and episode termination conditions are task dependent and

must be defined individually for each task.

For the take-off and hover task considered in this case study, the RPM control mode is used for a single agent. To further simplify the control problem, a single scalar action is applied to all four motors simultaneously, such that $P = P_0 = P_1 = P_2 = P_3$, reducing the action space to a single normalized dimension. This simplification is well-suited to the hover task, where symmetric thrust is sufficient to achieve vertical stabilization when no noise or wind disturbances are present. As a consequence of this symmetry, the lateral position (x, y), lateral velocities, and all angular velocities remain close to zero throughout the episode and carry no informative signal for control. The task therefore effectively reduces to a one-dimensional altitude control problem, where only the vertical position z, the vertical velocity v_z, and recent motor outputs are the relevant state variables.

4.3 Results

As an initial test, a PPO agent was trained with default parameters on the take-off and hover task using the HoverAviary environment. The goal is for a single drone to reach a predetermined target altitude of 1 meter above its starting position and stabilize. Each episode runs for a maximum of 8 seconds, corresponding to 240 timesteps, truncating early if the drone exceeds an altitude of 2 meters. The observation space is as described in equation (6), normalized before being passed to the policy, and the action space is reduced to a single normalized scalar by applying the same RPM value to all four motors simultaneously, as described in Section 4.2.

The HoverAviary built-in reward function is designed to incentivize the drone to reach and maintain the target position $\mathbf{p}_{target} = [0, 0, 1]$ by penalizing its distance from the goal. At each timestep, the reward is computed as:

$$r = \max\big(0,\ 2 - \|\mathbf{p}_{\text{target}} - \mathbf{p}\|^4\big) \tag{9}$$

where $\mathbf{p}$ denotes the current position. The reward function curve can be seen in Figure 1. The drone is initialized at $[0, 0, 0]$, corresponding to an initial distance of 1 meter from the target, at which point the reward evaluates to 1. The reward attains its maximum value of 2 when the drone is exactly at the target and decreases rapidly with increasing distance due to the fourth-power term, becoming zero for distances greater than approximately 1.19 meters.

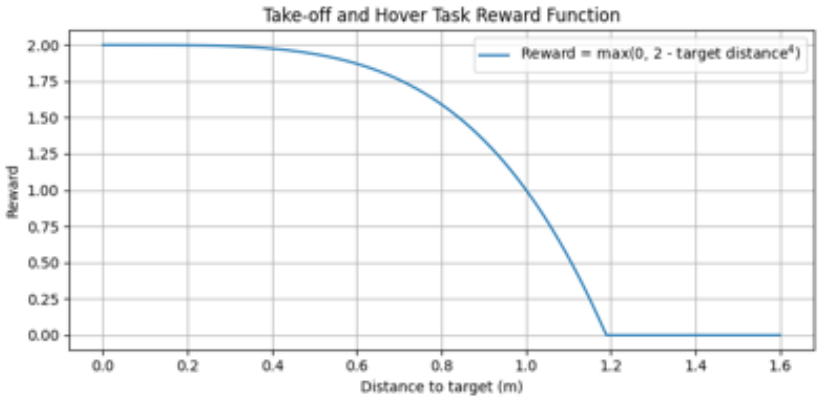


Figure 1: The take-off and hover task reward function $r = \max(0, 2 - \|\mathbf{p}_{\text{target}} - \mathbf{p}\|^4)$ as a function of distance to the target position.

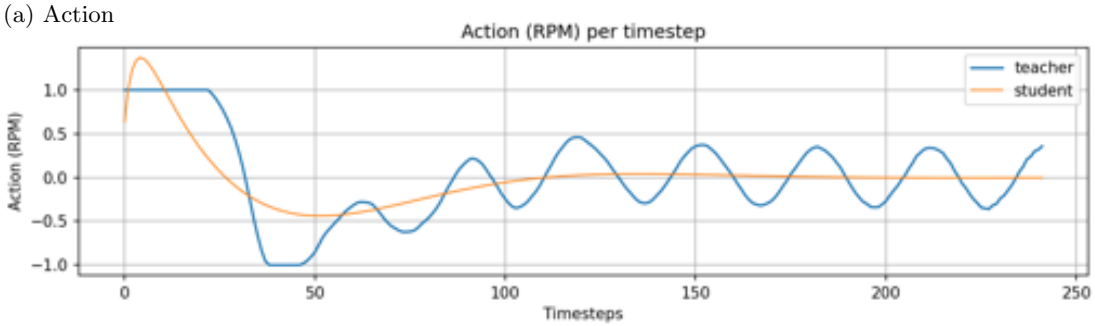
The PPO model was trained for approximately 10^7 steps, equal to approximately 40,000 episodes, after which it achieved a mean episode reward of approximately 474. Training was straightforward overall, and the trained teacher successfully completes the task across episodes.

However, while the PPO teacher reliably reaches the target altitude, it exhibits minor oscillations around it during hovering phase of the task. More notably, the motor outputs produced by the PPO teacher exhibit persistent fluctuations in RPM values rather than converging to a stable hover thrust. This behavior is consistent with the properties of the reward function. While the reward provides a strong incentive for accurate positioning near the target, it is relatively flat in a neighborhood around the goal, as visible in Figure 1, the reward changes slowly for distances below below 0.4 meters, meaning the policy receives nearly identical rewards whether it hovers precisely at the target or within a moderate radius of it. As a result, the PPO teacher's Q-function does not distinguish sharply between precise and approximate hovering in this region, leaving the policy free to produce variable motor outputs without incurring significant return loss. Furthermore, the reward offers limited feedback when the drone is further away from the target, dropping to zero beyond approximately 1.19 meters, providing no gradient signal to guide the policy in these regions. As we discuss below, the flatness near the target has implications for the stability of the subsequent distillation.

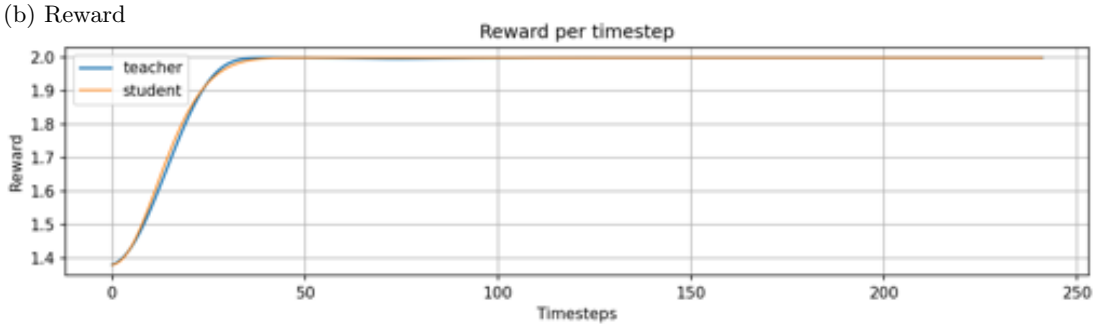
The trained PPO teacher was subsequently distilled into a symbolic student policy using the GM-DAgger procedure, following the same distillation pipeline used in the benchmark experiments (Section 5.2.1), with minimal tuning of the number of DAgger iterations, total timesteps, maximum expression size (`maxsize`), and arithmetic operators. The symbolic search space was primarily restricted to simple arithmetic operations (addition, subtraction, multiplication, and division). The number of DAgger iterations was found to have a meaningful effect on the quality of the resulting student: with shorter iteration counts, the distilled policy learned to fly upward but was unable to stabilize at the target altitude. With longer iterations, a more stable flight pattern emerged, with clear deceleration as the drone approached the target and consistent hover behavior once it arrived. All results reported below use the longer iteration setting, specifically fixed to 10.

A distilled symbolic student policy trained using the GM-DAgger objective can be seen in Figure 2 (a), where both the PPO teacher and the student successfully complete the task and receives near-equal rewards. However, despite their similar performance, the two policies differ considerably in how they achieve it. While the PPO teacher exhibits significant oscillations in its motor output throughout the episode, the student produces a far smoother action curve, suggesting that the distillation procedure has found a cleaner generalization of the underlying task rather than simply imitating the PPO teacher's specific behavior. The distance and velocity curves further illustrate this: the student approaches the target slightly slower than the PPO teacher, but stabilizes more precisely upon arrival, with the velocity converging closer to zero and less residual fluctuations around the hover point.

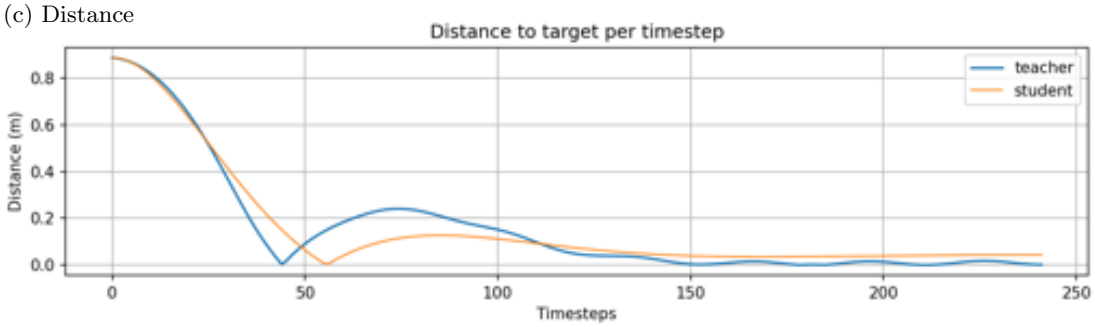
(a) Action



(b) Reward



(c) Distance



(d) Velocity

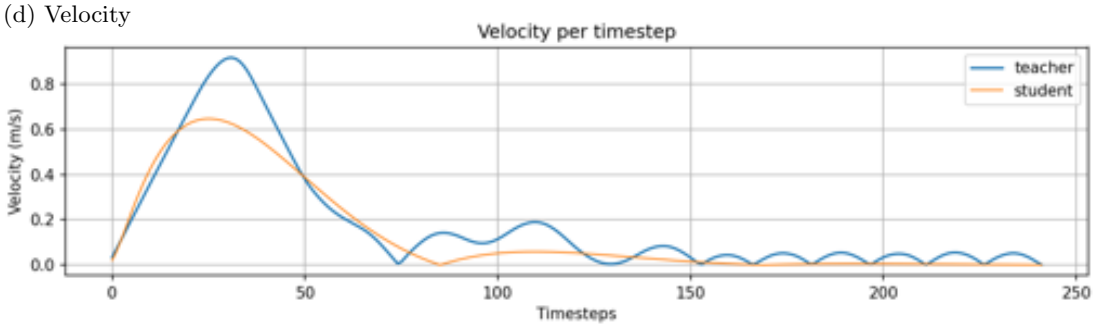


Figure 2: Comparison of the PPO teacher and symbolic student policies on the take-off and hover task, distilled using 5,000 DAgger timesteps, `maxsize` 20 and simple operators. (a) Motor action (RPM) per timestep. (b) Reward per timestep. (c) Distance to target per timestep. (d) Vertical velocity per timestep.

The distilled student policy takes the form:

$$P_t = 0.6204503 \cdot (x_{26} - 1.0993544 \cdot x_2 - x_8) + 0.71136916 \tag{10}$$
$$= 0.6204503 \cdot (P_{t-1} - 1.0993544 \cdot z - v_z) + 0.71136916 \tag{11}$$

where P_t is the motor RPM action at timestep t, z is the current altitude, and v_z is the current vertical velocity. The expression uses only three of the 27 available observation dimensions, which is consistent with the task simplification described in Section 4.2: since all four motors are driven identically and no lateral disturbances or noise are present, altitude and vertical velocity are the only variables relevant to the control task, and the previous motor output P_{t-1} may provide a form of action smoothing.

The structure of equation (10) closely resembles a discrete time Proportional-Derivative (PD) controller [49]. A PD controller computes a control output as a weighted sum of the current error and its rate of change with the proportional term correction deviations from the target and the derivative term dampening oscillations by reaching the speed of change. In the context of altitude control, the error is the deviation from the target altitude $z_{\text{target}} - z$, and its rate of change is approximated by the vertical velocity v_z. The distilled expression mirrors this structure: the $-1.0993544 \cdot z - v_z$ term acts as a combined proportional-derivative signal, penalizing both the current altitude and its rate of change, while the constant 0.71136916 acts as a bias term approximating the hover thrust needed to counteract gravity. The scaling by P_{t-1} is not present in a standard PD controller, and may introduce a form of action smoothing that further dampens the fluctuations observed in the PPO teacher's motor outputs.
That symbolic regression recovered this physically meaningful control structure without any prior knowledge of control theory is a compelling interpretability result: the distilled policy is not only compact but directly inspectable in a way the DRL teacher policy is not, and its structure aligns naturally with classical control design principles that engineers and practitioners would immediately recognize.

A notable difference between the PPO teacher and student concerns action constraints. While the PPO teacher's actions are clipped to the interval $[-1, 1]$ during both training and evaluation, this restriction is not inherited by the symbolic student, whose outputs can exceed this range as visible in the action panel of Figure 2.(a). This is a general property of the distillation approach where hard constraints imposed on the teacher through post-processing or clipping are invisible to the student during training, since the student only observes the teacher's already-clipped outputs within the visited state distribution. In safety-critical applications this has important implications, as it means that any required action bounds must be explicitly enforced on the deployed symbolic policy rather than assumed to be inherited from the teacher.

While the main result above presents a clean and interpretable outcome, it is important to note before proceeding to the remaining experiments that the distillation procedure exhibited considerable run-to-run variance, which affects the interpretation of all subsequent results. This is illustrated in Figure 3, which shows the action curves of five repeated distillation runs under identical configurations across four complexity levels. Even for fixed complexity and `maxsize`, the resulting student policies vary considerably in their action curves. The corresponding reward, distance, and velocity curves are shown in Figure 15 in Appendix A, where the variability is equally apparent across all metrics. While some runs converge to expressions resembling the PD-like controller recovered in the main experiment (equation (10)), others produce qualitatively different behavior, underscoring the stochastic nature of the underlying symbolic regression procedure. Notably, however, for higher `maxsize` settings, selecting the student with the best score, accounting for both loss and expression complexity, tends to recover policies with action curves similar to the PD-like controller.

Figure 3: Actions RPM for five repeated distillation runs under identical configurations across four complexity levels (13, 15, 21, 23), using 20,000 timesteps and `maxsize` 30 with simple operators.

We attribute this variance primarily to the flatness of the reward function near the target, as discussed above. As a result, the GM-DAgger loss offers weak signal to guide the symbolic search toward one expression over another, leaving the evolutionary search in PySR [10] free to converge to different local optima across runs. In this sense, the variance observed here is therefore not purely a property of symbolic regression itself, but is amplified by the reward design of the underlying task. A denser reward signal would likely reduce this variability by providing sharper Q-value differences between near-optimal and suboptimal student behaviors, giving the distillation procedure a clearer objective to optimize. This points to reward function design as an important consideration not only for training the teacher, but for the reliability of the subsequent distillation.

Apart from the above, multiple experiments were carried out, varying the total number of DAgger timesteps, the maximum expression size (`maxsize`), and the set of available operators, including more complex unary operators such as sin, cos, $\sqrt{}$, exp, and log.
Given the variance described above, the results of these experiments should be interpreted as broad tendencies observed across repeated runs rather than precise or reproducible outcomes. While the directional trends described below were generally consistent across runs, the degree to which they manifested varied considerably, and individual runs could deviate substantially from the overall pattern.

With this caveat in mind, Figure 4 shows the effect of increasing expression complexity on the student's action curve across six complexity levels. A clear trend emerges where at low complexity, the student produces smooth, generalized action curves that achieve strong task performance but diverge considerably from the PPO teacher's specific action pattern. As complexity increases, the student actions progressively converge toward the PPO teacher's oscillating behavior, with the fidelity gap from equation (3) decreasing monotonically while task performance remains roughly equivalent. This reflects an interpretable trade-off where low-complexity expressions generalize the task well and may even produce smoother and more stable control than the teacher, whereas higher-complexity expressions become increasingly faithful imitations

of the teacher, reproducing its fluctuations at the cost of interpretability. The same trend is further visible in the corresponding reward, distance, and velocity curves shown in Figure 16, 17, and 18 respectively in Appendix A, where despite the diverging action profiles, cumulative rewards remain near-equal across complexity levels, while distance and velocity curves show progressively closer alignment with the PPO teacher as complexity increases.

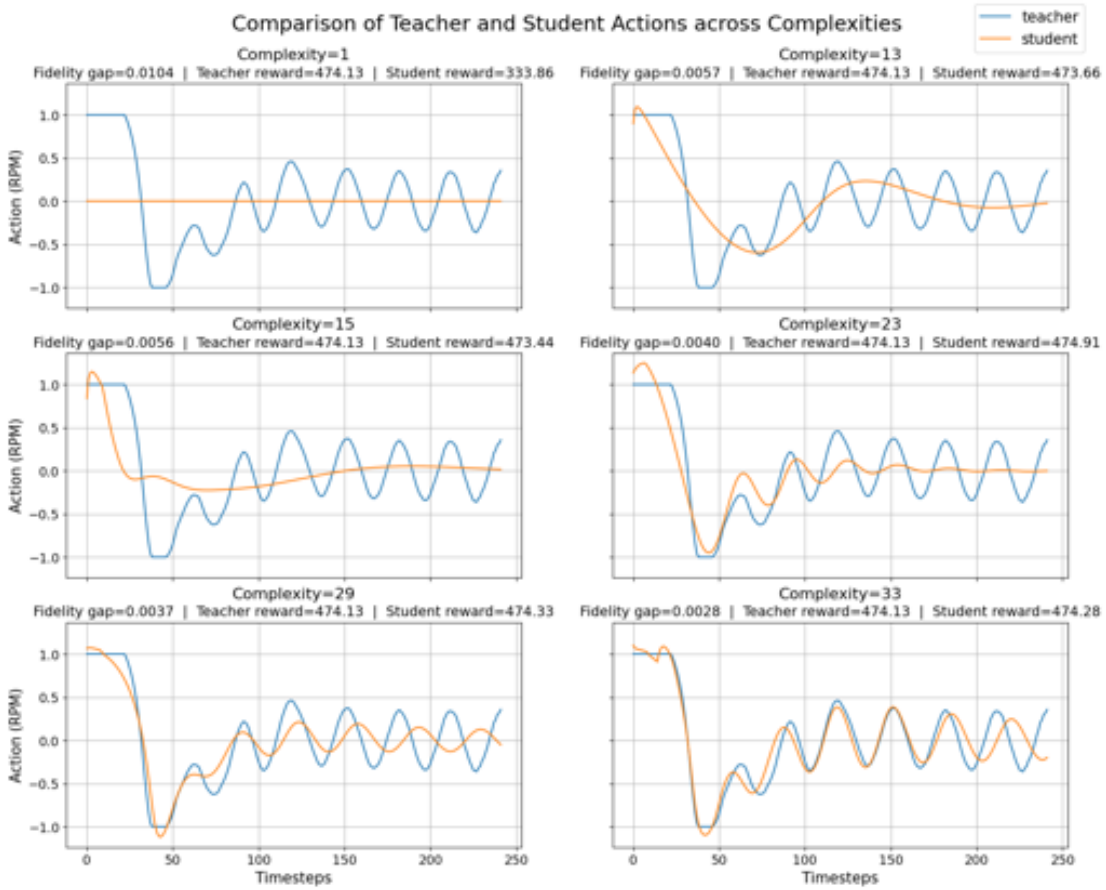


Figure 4: Comparison of teacher and student action curves across six expression complexity levels, using 20,000 timesteps, `maxsize` 40, and simple operators.

Table 2 shows the symbolic expressions recovered at each complexity level. At low complexity, the expressions rely primarily on x_2, x_8, and x_{26}, which corresponds to the previous motor output and a combination of state variables closely related to those identified in the PD-like expression in equation (10), suggesting that the distillation procedure consistently identifies the same set of physically relevant variables at low complexity. As complexity increases, additional historical action buffer variables containing recent motor outputs such as x_{17}, x_{19}, x_{18}, and x_{14} are progressively introduced, and the expressions become increasingly nested and nonlinear. While the fidelity gap decreases monotonically with complexity, the expressions become harder to interpret and the physical meaning of the additional variables is less clear. This further illustrates a trade-off in symbolic policy distillation where low-complexity expressions are more interpretable and may generalize the task better, while high-complexity expressions may more faithfully reproduce the teacher's behavior at the cost of transparency.

C	Expression	Fid	Reward
1	x_{26}	0.0104	333.86
13	$x_{26} - 0.083 \cdot \frac{x_{26}-1.376}{x_2+x_8} - 0.113$	0.0057	473.66
15	$x_{26} - 0.070 \cdot \left(x_{17} + 1.610 + \frac{x_{26}-1.528}{x_2+x_8}\right)$	0.0056	474.91
23	$0.966 \cdot \left(x_{26} + \frac{0.134}{x_{26}^2+x_2+x_8} - 0.135 \cdot x_2(x_8 + x_{19} + 0.946)\right)$	0.0040	474.91
29	$\frac{0.121}{x_2+0.535x_{26}^2} + x_{26} - 0.122\left(x_{26}^3 + x_2 + x_2(2.263x_8 + x_{18})\right)$	0.0037	474.33
33	$\frac{0.125}{x_2+0.588x_{26}^2} + x_{26} - 0.125\left(x_2 + x_{26}^2(x_{26} - x_{14}x_{12}) + x_2(2.312x_8 + x_{18})\right)$	0.0028	474.28

Table 2: Symbolic expressions recovered at six complexity levels, using 20,000 DAgger timesteps, `maxsize` 40, and simple operators. C denotes expression complexity and Fid denotes the fidelity gap between the PPO teacher and student policies.

Figure 5 further illustrates the effect of `maxsize` on this trade-off, showing episode reward as a function of expression complexity for four `maxsize` settings. For each `maxsize` setting, the minimum expression complexity required to find a well-performing student tends to increase with `maxsize`, consistent with a larger symbolic search space requiring more expression nodes before good solutions are reliably discovered. Importantly, increasing `maxsize` beyond what the task requires does not reliably improve either task performance or fidelity, and can lead to unnecessarily long expressions that are harder to interpret. This suggests that `maxsize` should be chosen conservatively, set only as large as needed to achieve acceptable performance, and increased further only when higher fidelity to the teacher is a specific requirement.

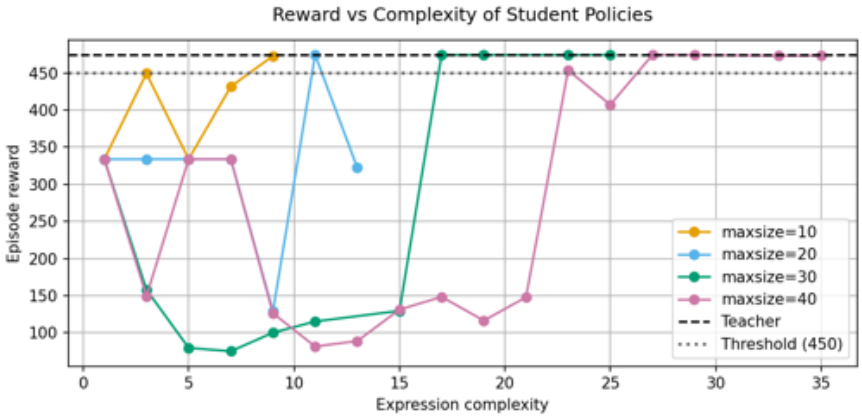


Figure 5: Episode reward as a function of expression complexity for four `maxsize` settings, using 5,000 timesteps with simple operators.

This is further supported by Figure 6, which shows the minimum expression complexity required to exceed a reward threshold of 450 across repeated runs for each `maxsize` setting. The mean complexity required increases with `maxsize`, confirming that larger search spaces require more complex expressions before well-performing students are reliably found. Notably, for `maxsize`=10, several runs failed to find a policy exceeding the threshold at any complexity level, suggesting that too small a `maxsize` can be overly restrictive.
Together, these results suggest that `maxsize` should be chosen conservatively, set only as large

as needed to achieve acceptable performance.

Figure 6: Minimum expression complexity achieving a reward above 450 across 10 distillation runs for four `maxsize` settings, using 5,000 timesteps with simple operators. Each dot represents one distillation run and the horizontal bar shows the mean. For `maxsize`=10, only 4 out of 10 runs exceeded the threshold.

Regarding the more complex unary operators, their inclusion generally did not improve performance for higher-complexity expressions compared to restricting the search to simple arithmetic operators. For lower-complexity expressions, however, the inclusion of operators such as *sin* produced action curves that diverged more substantially from both the PPO teacher and the simple-operator students, without a corresponding improvement in task performance. This suggests that for this particular task, the added expressiveness of complex unary operators introduces unnecessary search space complexity without benefit, and that simple arithmetic operators are sufficient to capture the underlying control structure.

The effect of varying the total number of DAgger timesteps was less straightforward to interpret. Experiments were conducted across a range of timestep counts between 2500 and 20,000, but the high degree of run-to-run variance made it impossible to isolate the effect of timestep count from the inherent stochasticity of the symbolic regression procedure. Any difference in performance or fidelity between timestep settings could equally be attributed to variance in the distillation runs as to the number of timesteps itself, and no consistent directional trend was observed that held reliably across repeated experiments. We therefore refrain from drawing conclusions about the effect of timestep count on distillation quality in this setting, though the absence of a clear benefit from longer rollouts suggests that lower timestep counts may be preferable in practice to reduce computation time.

4.4 Case Discussion

The drone case study demonstrates that the SPID framework generalizes beyond the environments considered in the original paper, successfully recovering compact and interpretable controllers from DRL policies trained in a simple, but more realistic physics-based environment. The distilled PD-like expression not only matches the teacher's task performance but actually surpasses it in control stability, producing smoother hover behavior from just three variables (altitude, vertical velocity, and the previous action) out of the 27 available, consistent with the task simplification described in Section 4.2. The fact that symbolic regression recovered a physically meaningful control structure without any prior knowledge of control theory is a strong result for interpretability. It also suggests that distillation can serve as a kind of implicit feature selection tool, helping to identify which state variables the teacher actually relies on for its decisions. In this case, symbolic regression selected altitude, vertical velocity, and previous action out of

27 available dimensions without any explicit guidance toward these variables. Notably, this selection aligns with classical control theory for altitude regulation, where these are precisely the variables one would expect a well-designed controller to rely on [49]. This correspondence between the distilled expression and established control principles provides additional support for the interpretability claim: the symbolic policy is not only compact and inspectable, but its structure is independently meaningful.

One important limitation to keep in mind is that the teacher's action constraints are not automatically transferred to the student. The teacher's outputs are clipped to $[-1, 1]$ before being stored as distillation targets, but this does not make the student aware of the bound itself, as the symbolic expression has no built-in knowledge of why the training data happened to fall within that range. As a result, the student can exceed the teacher's action bounds when deployed. Any required action bounds must therefore be explicitly enforced on the deployed symbolic policy, for example through direct clipping of the symbolic expression at deployment, or through constraint terms added to the distillation loss. This issue is likely not unique to this setting, but may arise more broadly in symbolic policy distillation wherever hard action constraints are imposed on the teacher through post-processing rather than as part of the learned behavior itself, and is worth considering carefully in safety-critical deployments contexts.

The most practically relevant finding from this case study is how sensitive the distillation procedure is to the reward function used to train the teacher. The flat reward landscape near the target means the teacher's Q-function is also relatively undifferentiated in that region, which in turn weakens the distillation signal and likely contributes to the considerable run-to-run variance observed in Figure 3. However, it is worth noting that symbolic regression via PySR introduces its own stochasticity through its evolutionary search procedure, independently of the reward landscape. Those two sources of variance have different remedies: a denser reward signal would likely reduce variance driven by weak distillation signal, while variance stemming from the symbolic search itself may require different mitigation strategies, though in practice it is difficult to fully separate their contributions from the results observed here.

Additionally, the complexity and `maxsize` experiments highlight a trade-off that is worth being deliberate about. Lower-complexity expressions tend to generalize the task well and can actually be more stable than the teacher, but they diverge from its specific behavior. Higher-complexity expressions track the teacher more closely but become harder to interpret and can inherit its less desirable behaviors, such as the oscillations seen in Figure 2a. Neither is universally better, but instead depends on whether the priority is a clean, interpretable controller or a faithful approximation of the teacher. What is clear is that `maxsize` should be set conservatively, as pushing it higher tends to increase expression complexity and variance seen in Figure 6 without reliably improving either performance or fidelity.

Finally, it is worth noting that the task considered here is deliberately simple and noise-free. The drone operates in a controlled simulation environment with no sensor noise, wind, or other external disturbances, and the symmetry of the task means that lateral dynamics are effectively irrelevant. This simplicity was an intentional design choice that aided the recovery of a compact and interpretable symbolic expression, and while this makes the setting well-suited for demonstrating the distillation approach, it also means that the distilled PD-like expression has never been exposed to any form of perturbation during either training or evaluation. The simplicity of the recovered expression, which relies only on altitude, vertical velocity, and the previous action, suggests it may be brittle in the face of disturbances that push the drone off its vertical trajectory, as it has no mechanism for correcting lateral deviations. Real-world deployment would therefore likely require either a more complex symbolic policy that accounts for the full state space, or a task setup that introduces noise and disturbances during training to encourage the distilled policy to develop more robust control behavior. This is an inherent limitation of the sim-to-real gap in symbolic policy distillation, and one that would need to be addressed before considering deployment in a physical system.

5. Case Study: Using a Symbolic Policy to Stabilize Blood Glucose

5.1 Introduction

Type 1 diabetes mellitus (T1DM) is a chronic disease characterized by the body's inability to produce sufficient insulin. Without adequate insulin, glucose cannot be properly regulated, leading to elevated blood glucose (BG) levels. Individuals with T1DM therefore require lifelong insulin therapy, commonly administered through insulin injections or insulin pumps, in order to maintain BG within a safe range [45].

Maintaining BG within this range is essential because both elevated and reduced glucose levels can have serious consequences. Chronic high BG, referred to as hyperglycemia, is associated with long-term complications such as cardiovascular disease, nerve damage, kidney failure, and vision impairment [45]. In severe acute cases, hyperglycemia can lead to diabetic ketoacidosis, a potentially life-threatening condition caused by the accumulation of acids in the blood [12]. Conversely, excessively low BG, referred to as hypoglycemia, can occur when too much insulin is delivered relative to the body's needs. Hypoglycemia may cause confusion, dizziness, and loss of consciousness, and in severe cases can lead to seizures or death if not treated immediately [27].

Insulin dosing is therefore a safety-critical control problem. The controller must deliver enough insulin to prevent prolonged hyperglycemia, while avoiding excessive insulin delivery that may cause acute hypoglycemia. This trade-off is complicated by nonlinear glucose-insulin dynamics, delayed insulin action, uncertain meal effects, physical activity, stress, illness, and substantial inter-patient variability. As a result, effective insulin control requires repeated decisions under uncertainty, where both the magnitude and timing of insulin delivery are important.

This makes automated insulin delivery a relevant real-world case for studying interpretable RL. DRL is attractive because it can learn patient-specific control policies through interaction with a simulated patient environment, without requiring the full control strategy to be manually specified. However, the use of neural network policies introduces a major limitation in this setting: even if a DRL controller performs well in simulation, its dosing strategy may be difficult to inspect, explain, or constrain. In a medical control problem, average performance alone is insufficient, the controller should also be transparent enough to assess whether its behavior is clinically plausible and whether unsafe dosing patterns may occur.

This motivates the use of symbolic policy distillation in the diabetes-control case study. Rather than treating the trained DRL policy as the final controller, we use it as a teacher and distill its behavior into a symbolic student policy. The resulting expression can be inspected directly, making it possible to analyze how BG, Insulin on Board (IOB), meal information, or previous actions influence the recommended insulin dose. This is valuable both as a way to understand the learned DRL policy and as a potential controller representation in its own right.

Symbolic policies are also closer to the explicit rule-based structure used in many insulin-delivery algorithms than neural network policies. While symbolic distillation does not by itself guarantee safety, an analytical dosing rule is easier to inspect, compare, restrict, and test for undesirable behavior than a black-box neural controller. In this thesis, the diabetes-control case study is therefore used to examine whether SPID can transfer useful behavior from DRL into compact symbolic insulin-dosing policies, while improving transparency in a safety-critical medical setting.

To place this control problem in context, the following section first describes how insulin

delivery is performed in current diabetes management, and how this has motivated increasingly automated treatment systems.

5.1.1 Automated Insulin Delivery Systems

Classical management of T1DM requires patients to continuously monitor their BG levels and administer insulin according to daily physiological needs. Traditionally, this has been done through multiple daily injections (MDI), while insulin pump therapy, also known as continous subcutaneous insulin infusion (CSII) provide more precise and flexible insulin administration [8]. Unlike traditional injections, insulin pumps continuously deliver small basal doses of insulin and allow for carefully calculated bolus doses at mealtimes [6]. Compared with MDI, CSII has been associated with improved glycemic control and reduced severe hypoglycemia in several studies and meta-analyses [29].

However, determining the correct amount and timing of insulin delivery remains a difficult task. BG dynamics are affected by uncertain and highly individual factors, including meal composition, physical activity, stress, illness, and patient physiology [30]. In addition, insulin does not act immediately after delivery, creating a delay between insulin administration and its effect on BG [14]. Hence, patients and insulin delivery systems must continuously make decisions under uncertainty while balancing the risks of hyperglycemia and hypoglycemia.

To address these challenges, insulin pumps have increasingly been integrated with continuous glucose monitoring (CGM) systems and automated insulin control algorithms. Such systems are commonly referred to as Artificial Pancreas Systems (APS), and represent one of the latest developments in diabetes treatment [47]. An APS is typically formulated as a fully closed-loop (FCL) system consisting of a CGM sensor, an insulin pump, and a control algorithm that adjusts insulin delivery in response to real-time glucose measurements. It is a FCL system because the controller only uses feedback from the system output to adjust its future actions.

In practice, however, most current systems are hybrid closed-loop (HCL) systems, as they can automatically adjust basal insulin delivery but still require manual input from the user, particularly meal announcements or carbohydrate estimates, to administer bolus insulin effectively [26]. The term hybrid therefore reflects that insulin delivery is partly automated through closed-loop feedback control, while part of the control process remains user-dependent.

From a control systems perspective, insulin delivery can be formulated as a dynamic feedback control problem, where insulin administration acts as the control input and BG concentration is the system output. The objective is to design a controller capable of maintaining glucose within a safe target range while adapting to changing physiological conditions and external disturbances. Several control strategies have been used in APS, including basal-bolus (BB) control, Proportional-Integral-Derivative (PID) control, fuzzy logic control, and model predictive control (MPC) [39]. These approaches have shown practical value and form the basis of many current automated insulin delivery systems.

Despite these advances, existing control methods still face important limitations. Many systems rely on simplified mathematical models, hand-designed rules, or predefined clinical parameters that may not fully capture personalized glucose-insulin dynamics. Moreover, most current systems still depend on external user inputs, such as carbohydrate intake, to handle rapid glucose fluctuations caused by meals or other disturbances. Accurately estimating carbohydrate intake is difficult and error-prone, which increases the cognitive burden on patients and may lead to unsafe dosing decisions. These limitations are further complicated by nonlinear glucose dynamics, delayed insulin effects, and substantial inter-patient variability.

DRL is emerging as a promising alternative for artificial pancreas control [47]. In contrast to fixed rule-based or model-based methods, DRL-based controllers can learn insulin dosing

strategies directly through interaction with a patient or simulated patient environment. This makes DRL relevant for handling uncertainty, delayed effects, and patient-specific variability.

5.1.2 Glycemic Control Using Reinforcement Learning

The development of artificial pancreas control algorithms has been accelerated by computer simulation, allowing *in-silico* development and evaluation of insulin control strategies before clinical testing. The UVA/Padova Type 1 Diabetes Simulator is an FDA-approved simulation model that represents patient-specific glucose-insulin dynamics and supports simulated meal scenarios. The simulator models the main subsystems involved in glucose production, consumption, and secretion, together with the dynamics of insulin absorption and degradation following insulin pump delivery[7].

However, the UVA/Padova simulator is a commercial model. Therefore, for accessibility and reproducibility, we use Simglucose, an open-source Python implementation based on the UVA/Padova simulator, commonly used for research[46]. Simglucose provides a cohort of 30 virtual patients, consisting of 10 adults, 10 adolescents, and 10 children.

These simulation environments have enabled the development of DRL methods for insulin control. In this setting, the controller learns an insulin dosing policy by interacting with a simulated patient environment and receiving feedback based on the resulting glucose response. DRL-based controllers have been studied in both HCL settings, where meal information is available, and FCL settings, where the controller relies only on CGM measurements. Compared with conventional control methods, DRL may be better suited to learning patient-specific dosing strategies under nonlinear dynamics, delayed insulin effects, and uncertain disturbances.

The challenge of glucose control using DRL has been addressed by several previous works, and has become a highly active area of research in recent years [48]. Existing approaches vary substantially in their choice of learning algorithm, simulator, state representation, action space, and reward formulation. Both off-policy and on-policy methods have been explored, including value-based methods, actor-critic algorithms, and direct policy-optimization approaches. Similarly, state spaces differ across studies, ranging from current glucose measurements alone to richer representations that include glucose history, insulin-on-board, meal announcements, time intervals, and patient-specific information. Action spaces may be discrete, corresponding to a finite set of insulin decisions, or continuous, allowing the agent to output insulin doses directly. Reward functions also differ considerably, but are generally designed to encourage normoglycemia while penalizing hypoglycemia, severe hyperglycemia, or excessive insulin delivery.

Recent examples illustrate this diversity. Fox et al. [15] use Simglucose to train a Soft Actor-Critic controller for closed-loop glucose regulation, considering both settings with and without meal announcements. Their work further incorporates transfer learning to improve sample efficiency and support patient-specific modelling. Viroonluecha et al. [43] evaluate both SAC and PPO-based controllers in the UVA/Padova simulator, reporting stronger performance for PPO, although the learned controllers still exhibit limited survival rates in some settings. Dénes-Fazakas et al. [13] likewise investigate PPO-based glucose controllers, comparing models trained with and without meal information. Their work evaluates several training and testing scenarios, including differences in the number of simulated training and evaluation days, and compares multiple reward formulations, again identifying PPO as one of the strongest-performing methods. Zhao et al. [47] also train a PPO-based controller for fully closed-loop glucose control without meal announcements, with particular emphasis on safety enhancements and training stability in Simglucose.

Although DRL provides a promising approach to adaptive insulin control, it also introduces important limitations, and show the task remains difficult. Moreover, DRL-based insulin controllers are subject to high safety constraints, and must be thoroughly tested and verified. Neural

network policies are generally difficult to interpret, making it unclear why a given insulin dose is selected in a specific state. This is problematic in a safety-critical medical context, where controller behavior should be transparent, reliable, and preferably verifiable. A DRL policy may achieve strong performance in simulation, but its black-box structure makes it difficult to inspect whether the learned behavior is clinically reasonable or safe under unseen conditions.

Rather than deploying the trained DRL policy directly, we aim to distill its behavior into a patient-specific symbolic policy. The purpose is to transfer the knowledge captured by the complex neural controller into a compact mathematical expression that is easier to interpret and analyze. We therefore train PPO-based DRL controllers for both hybrid closed-loop and fully closed-loop insulin control across individual adult virtual patients, and subsequently distill these policies into symbolic representations. The goal is to retain the adaptive potential of DRL while producing policies whose behavior and safety properties can be more directly inspected. Moreover, we expect the symbolic policies to aid in the understanding of the DRL algorithms and their behavior.

5.2 Methods

5.2.1 Benchmark

We compare our models against two baseline methods for control: BB and a PID-controller. BB reflects how a human-in-the loop would control their glucose levels, with manual inputs from the user at meal times. The PID controller is a common control algorithm in APS for fully closed-loop control, handling both a base-level of insulin and insulin management at meal times.

To evaluate our models and benchmarks, we use classical RL scores of rewards combined with insulin-controller specific evaluation metrics, which is described in Section 5.2.1.3.

5.2.1.1 Basal-Bolus Controller The BB controller represents an ideal case of how an individual with Type 1 Diabetes controls would manage their BG levels using an insulin pump. A BB-Controller maintains a base level (Basal) of insulin based on clinical treatment strategies recommendations by administering small, continuous insulin doses. However, the BB-Controller is not fully automatic, and require the user to manually input announcements for meals with estimated calorie intake (CHO) to counteract the rise in BG after meals (Bolus insulin).

The BB insulin dosage, u_t^{BB} is administered according to the Formula (12).

$$u_t^{BB} = bas + (c_t > 0) \cdot \left(\frac{c_t}{CR} + cooldown \cdot \frac{b_t - b_g}{CF} \right) \tag{12}$$

Where c_t is the carbohydrates of the meal, and b_t is the current measure of CGM with b_g being the target glucose level. The basal-insulin rate bas, the correction factor (CF) and the carbohydrate to insulin ratio (CR) are patient-specific values determined by a physician made available by the environment. [11, 15, 46]

In an ideal basal-bolus setting, meal timing and carbohydrate amount would be known exactly. However, this is an optimistic assumption, since real patients may announce meals slightly before or after eating and may misestimate carbohydrate content. So for the results we instead used basal-bolus human (BBH) which introduced noise in both the announced meal time and carbohydrate estimate. The patient-specific CF, CR and bas values were kept fixed at their simulator-provided values, so that BBH isolates the effect of imperfect meal reporting rather than errors in prescribed treatment parameters.

5.2.1.2 PID Controller A PID controller operates by setting the control variable, u_t^{PID}, to the weighed combination of three terms:

$$u_t^{PID} = k_P P(b_t) + k_I I(b_t) + k_D D(b_t) \tag{13}$$

In such a way that the current BG measure b_t remains close to a target value b_t. The terms are respectively the proportional term $P(b_t) = \max(0, b_t - b_g)$, which control the distance to the target value, the integral term $I(b_t) = \sum_{j=0}^{t}(b_j - b_g)$ that corrects long-term deviations from the target, and lastly the derivative term $D(b_t) = |b_t - b_{t-1}|$, the rate-of-change as a basic estimate for the future. The weights k_P, k_I, k_D and the target value b_g are hyperparameters. The target value is set to $b_g = 140\ mg/dL$, to ensure BG-levels stay within a safe range, and the risk of hypoglycemia is minimized. The weights are obtained from [15], which have determined the weights by grid-searching optimal values on a per-patient level.

5.2.1.3 Performance Evaluation Metrics In simulated artificial pancreas experiments, controller performance is commonly evaluated using CGM-derived glycemic metrics, stemming from clinical CGM defined metric standards. The primary metric is Time in Range (TIR), defined as the percentage of time BG remains within the normal target range 70–180 mg/dL. Safety is assessed using Time Below Range (TBR), typically glucose <70 mg/dL (level 1), and severe hypoglycemia, often glucose <54 mg/dL (level 2). Hyperglycemia is measured using Time Above Range (TAR), glucose >180 mg/dL (level 1), with very high or severe hyperglycemia commonly defined using thresholds such as >250 mg/dL (level 2). These metrics were standardized by the Advanced Technologies & Treatment for Diabetes Congress in 2019 [5].

Overall, the control objective was to minimize glycemic risk indices while maximizing time spent in the normoglycemic range, commonly expressed as TIR. Particular emphasis was placed on reducing TBR relative to TAR, as hypoglycemia represents a more immediate safety concern. Consequently, when a trade-off between hypo- and hyperglycemia is unavoidable, mild hyperglycemia is considered preferable to hypoglycemia, as described by current clinical standards. Current clinical guidance recommends a TIR of at least 70% of readings while limiting level 1 TBR to less than 4% and level 2 TBR to less than 1%. Although hyperglycemia should also be minimized, its risk profile is generally less acute than that of hypoglycemia; accordingly, level 1 TAR should be kept below 25% of readings, and level 2 TAR should remain below 5% [5].

The learned insulin dosing policies were evaluated over N independent simulated episodes. Each episode corresponds to one complete simulated control period for a single patient and has observed length $T_e \leq T$, where $T = 480$ for a 24-hour simulation with a 3-minute sampling interval. Episodes with $T_e < T$ were treated as critical failures and excluded from the computation of glucose-control and insulin-use metrics, but included in the Critical Failure Rate (CFR). Although the default Simglucose environment terminates only when the internal BG value leaves the range 10 - 600 mg/dL, our custom wrapper imposed stricter safety bounds and terminated episodes when CGM left the interval 40 - 400 mg/dL.

For completed episodes, CGM-derived glucose metrics were computed as the percentage of valid glucose samples falling within clinically defined ranges:

$$\begin{aligned}
\text{TBR}_{II} &= 100\mathbb{E}\left[\mathbb{I}(b_{e,t} < 54)\right], \\
\text{TBR}_{I} &= 100\mathbb{E}\left[\mathbb{I}(b_{e,t} < 70)\right], \\
\text{TIR} &= 100\mathbb{E}\left[\mathbb{I}(70 \leq b_{e,t} \leq 180)\right], \\
\text{TAR}_{I} &= 100\mathbb{E}\left[\mathbb{I}(b_{e,t} > 180)\right], \\
\text{TAR}_{II} &= 100\mathbb{E}\left[\mathbb{I}(b_{e,t} > 250)\right].
\end{aligned}$$

Here, $b_{e,t}$ denotes the CGM-measured BG concentration at time step t in episode e, and the expectation denotes the empirical mean over all valid glucose samples from completed evaluation episodes. Thus, TBR_I and TBR_{II} quantify level 1 and level 2 hypoglycemia, while TAR_I and

TAR_{II} quantify level 1 and level 2 hyperglycemia.

Episodes that terminated before the full evaluation horizon were treated as critical failures and excluded from the computation of TIR, TBR, TAR, and insulin-use metrics. This was done because early terminated episodes contain fewer glucose samples and could otherwise give misleadingly favorable range-based metrics. For example, an episode that terminates shortly after leaving the safe glucose range may still contain many earlier in-range samples, artificially inflating TIR. Such failures were therefore captured separately through the CFR. An episode was classified as a critical failure if it terminated before the intended horizon,

$$F_e = \mathbb{I}(T_e < T),$$

and the CFR was computed as

$$\text{CFR} = \frac{100}{N} \sum_{e=1}^{N} F_e.$$

Thus, CFR reports the percentage of evaluation episodes that terminated early due to unsafe glucose excursions.

Lastly, for all policies, we also report the average total reward across the N simulated episodes.

5.2.2 PPO Controller and Environment Interface

After defining the Simglucose simulation setup, the next step was to specify the RL interface used by the PPO controller, where the PPO-model was chosen due to its reported performance in glucose control, as described in other work [15, 15, 43]. Interface setup included defining the state representation available to the policy, the continuous insulin action space, normalization choices, meal-announcement handling, and safety-related modifications applied through the custom wrapper.

PPO was chosen because it is a widely used on-policy actor-critic algorithm for continuous-control problems and provides a practical balance between stability, implementation simplicity, and empirical performance. Its clipped surrogate objective limits the size of policy updates, which is useful in insulin-control settings where unstable policy updates may lead to unsafe dosing behavior.PPO is also a relevant choice in the context of artificial pancreas research, as several recent glucose-control studies have used PPO or PPO-based extensions in Simglucose or UVA/Padova-based simulation environments[47]. This makes PPO a relevant teacher algorithm for the diabetes-control case study, both as a strong DRL baseline and as a representative choice within recent simulation-based insulin-control research.

To adapt PPO to the insulin-control task, the agent was not connected directly to the raw Simglucose environment. Instead, it interacted with Simglucose through a custom Gymnasium wrapper that defined the observation space, action space, meal-information interface, insulin scaling and logging used in the study.

Two related controller interfaces were evaluated. The first was a meal-informed control setting, in which the policy received explicit meal-related information in addition to CGM feedback. This setting resembles a hybrid closed-loop (HCL) formulation, since insulin delivery is adjusted using glucose feedback while meal announcements and carbohydrate estimates are provided to support bolus-like responses. The second setting removed all meal information and therefore represented a fully closed-loop (FCL), meal-uninformed controller. Henceforth, these settings are referred to as the HCL and FCL settings, respectively.

For the HCL setting, the policy observation consisted of five variables: current CGM-measured BG b_t, time since the previous meal τ_t, estimated insulin-on-board IOB_t, a meal-warning indicator w_t, and announced carbohydrate amount c_t. These variables have different

physical units and numerical ranges, and were therefore normalized before being passed to PPO. This reduces differences in input scale and improves neural network training stability. The normalized state was defined as

$$s_t = [b_t/400,\ \tau_t/1440,\ \mathrm{IOB}_t/10,\ w_t,\ c_t/120].$$

Here, b_t is measured in mg/dL, τ_t in minutes, IOB_t in insulin units, and c_t in grams of carbohydrates. The meal-warning variable w_t was binary and was activated only within a 20-minute window before an announced meal. The announced meal size c_t was exposed to the policy only during this warning window, outside the window it was set to zero.

In the FCL setting, meal timing and meal size were removed from the policy input. Instead a short history of previous raw policy actions was added:

$$s_t = [b_t/400,\ \mathrm{IOB}_t/10,\ a_{t-5},\ a_{t-4},\ a_{t-3},\ a_{t-2},\ a_{t-1}].$$

Here, a_{t-k} denotes the raw normalized policy action from k previous control steps, before transformation into delivered insulin. This representation gives the controller limited temporal information about recent dosing decisions without exposing meal timing or carbohydrate information. The FCL policy therefore remains meal-uninformed, while still being able to condition its current action on recent insulin-delivery behavior. Meals still occurred in the simulator and were retained only for diagnostics and plotting.

In both the HCL and FCL settings, PPO produced a continuous scalar action $a_t \in [-1, 1]$. This raw action was transformed into a non-negative insulin control input using the exponential mapping

$$u_t = I_{\max} \exp\left(4(a_t - 1)\right),$$

where u_t denotes the insulin delivery rate passed to the simulator and $I_{\max}$ is the maximum allowed insulin delivery rate. The resulting insulin delivery rate was clipped to the interval $[0, I_{\max}]$. This action transformation was inspired by Hettiarachchi et al. [21] and was chosen to provide higher resolution for small insulin delivery rates, while still allowing larger rates when the policy output approaches the upper end of the action space. The maximum insulin delivery rate was set to $I_{\max} = 5.0$. This value was selected as a conservative controller limit rather than as a device-specific pump constraint. Although insulin pump settings may allow larger bolus deliveries, the lower cap was used to discourage abrupt insulin over-delivery during learning and evaluation. This choice was motivated by preliminary experiments in which less restrictive action limits often led the policy to repeatedly select the maximum action, increasing the risk of unsafe glucose excursions. The cap $I_{\max} = 5.0$ should therefore be interpreted as a safety constraint imposed on the controller, not as a general maximum capacity of insulin pumps

Meals were handled through explicit scenario modes rather than the default random Simglucose scenario. The implementation supported fixed meals, patient-specific Harrison-Benedict-based fixed meals, and semi-random Harrison-Benedict scenarios inspired by Viroonluecha et al.[43]. The Harrison-Benedict equation estimates basal energy expenditure from patient characteristics such as sex, age, height, and body weight, and was used here to define patient-specific nominal meal sizes[19].

Final training and evaluation therefore used the semi-random Harrison-Benedict scenario. Here, stochastic perturbations were applied to both meal timing and carbohydrate amount, introducing a controlled mismatch between announced and actual meals. The same meal dynamics were used in both controller settings: the HCL policy received meal-warning and carbohydrate information, while the FCL policy did not. This allowed the effect of meal information to be evaluated under comparable simulator conditions.

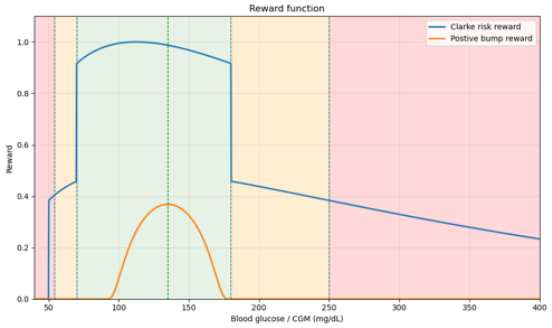


Figure 8: Clarke risk-based reward function used in the final diabetes-control experiments. The reward varies smoothly with glycaemic risk and gives reduced positive reward outside the target range, except in severe hypoglycaemia where the reward is set to zero.

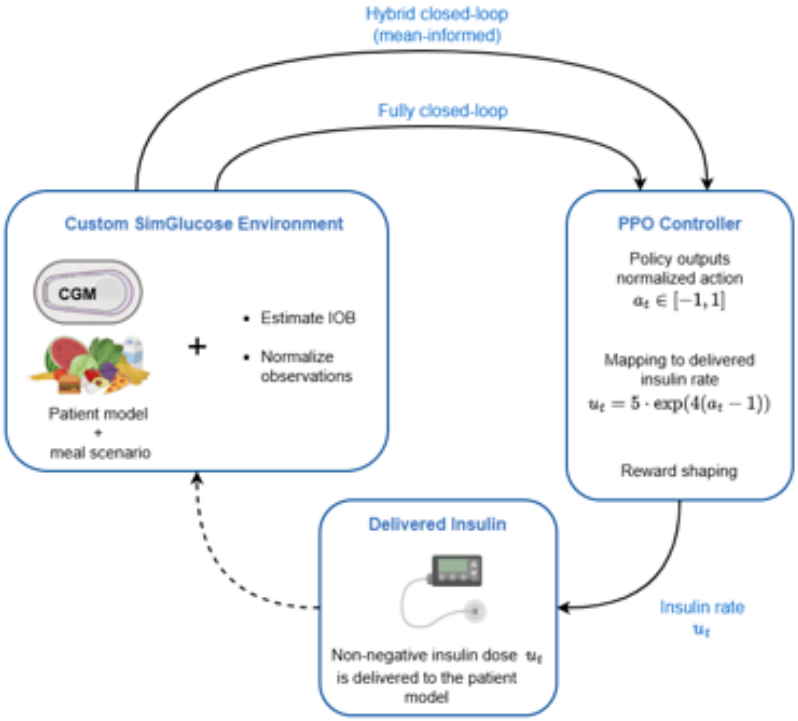


Figure 7: Overview of the custom Simglucose DRL interface used for insulin-control training.

Two implementation details are important for interpreting the results. First, IOB was not provided directly by Simglucose, but was estimated inside the wrapper using an exponential decay model with a default time constant of 55 minutes. The estimate was updated using the actually delivered insulin. This was done to account for delayed insulin action without explicitly encoding a full history of previous actions. The IOB estimate therefore served as a compact representation of residual insulin activity, reducing both state dimensionality and computational cost [4, 17].

Second, each episode was limited to 480 control steps, corresponding to 24 hours with a 3-minute sampling interval. The implementation also supported multi-day episodes, which were used later for robustness evaluation.

The reward function is a central design choice in RL-based BG control. Preliminary experiments tested several reward formulations, including rewards with strong negative penalties for hypoglycaemia, hyperglycemia, and terminal failure. Although these formulations encoded

safety objectives directly, they led to undesirable learning behavior. In particular, some policies learned to reach terminal failure early, thereby avoiding the accumulation of further negative reward over the remainder of the episode.

To avoid this behavior, the final experiments used a positive Clarke risk-based reward. Alternative positive rewards, including a compact bump reward, were also tested during development, but the Clarke reward produced the best overall performance in both the HCL and FCL settings, see Figure 8. It was therefore used for all final PPO training and subsequent symbolic distillation experiments.

The reward formulation is based on the Clarke glucose risk index introduced by Zhao et al.[47]. Let b_t denote the BG concentration at time t. The glucose risk is computed as:

$$h(b_t) = 1.509 \left(\log(b_t)^{1.084} - 5.381 \right), \tag{14}$$

The corresponding base reward is

$$r_{\text{risk}}(b_t) = \left(1 - \frac{10 \cdot h(b_t)^2}{180} \right)^2 \tag{15}$$

To increase the penalty for clinically unsafe glucose values, the final Clarke risk-based reward was defined as

$$R_{\text{risk}}(b_t) = \begin{cases} r_{\text{risk}}(b_t), & 70 \le b_t \le 180, \\ 0.5\, r_{\text{risk}}(b_t), & 50 \le b_t < 70 \text{ or } b_t > 180, \\ 0, & b_t < 50. \end{cases} \tag{16}$$

This reward assigns the highest values to glucose concentrations associated with low glycaemic risk, while reducing the reward outside the target range and assigning zero reward in severe hypoglycaemia, to reflect the biological background discussed in Section 5.2.1.3.

5.2.3 Training of the PPO Controller

A PPO controller was trained separately for each adult patient in the cohort (adult-001 - adult-010) for both the FCL and HCL control tasks. For each task, the same final hyperparameter configuration was used across all patients, such that differences in performance could be attributed primarily to patient-specific dynamics rather than patient-specific tuning.

Hyperparameter tuning was performed using Optuna[1], an automatic hyperparameter optimization framework. Optuna formulates tuning as a sequence of trials, where each trial evaluates one candidate set of hyperparameters. Based on the observed performance of previous trials, Optuna adaptively proposes new configurations, allowing the search to focus on more promising regions of the hyperparameter space. The hyper-parameter search space included the values as stated in Table 4 and 4. Some parameter setting were inspired by [47], which details the importance of adding policy entry, learning rate decay, advantage normalization, state normalization, as well as gradient clipping and orthogonal initialization for improved model convergence.

In the tuning of the PPO controllers, each trial was evaluated after 300,000 environment steps, corresponding to approximately 625 episodes, using the average total reward per episode as the optimization objective. For each patient, 15 trials were conducted, and the final hyperparameters were selected based on configurations that performed well across the full adult cohort rather than on a single patient. The final hyperparameters are shown in Table 3 and 4.

In addition to hyperparameter tuning, we compared training across different reward functions, including the Clarke risk reward and variants of the positive and negative bump rewards, as mentioned in Section 5.2.2. For both the FCL and HCL tasks, the Clarke risk reward generally produced better convergence and final performance.

Final PPO models were trained for 4 million environment steps, corresponding to approximately 8,300 simulated 24-hour episodes, starting at 00:00 and ending at 23:59. Training was computationally demanding, even when run on a computing cluster, training a single patient-specific PPO model often required more than 8 hours of wall-clock time. This also limited the extent of the hyperparameter search. During training, intermediate evaluations were performed every 100,000 steps. At each evaluation point, model performance was assessed using average reward, TIR, TAR, TBR, and CFR. The final model checkpoint was selected according to the highest average reward, as the reward typically reflected a balance between maintaining high TIR and low CFR.

The learning dynamics varied substantially across patients. In many cases, reward increased early in training as the model learned to deliver basal insulin. However, once the policy began to progress further into episodes and encounter meals, the learning curve often flattened or temporarily decreased, as the controller had to learn to respond to rapid glucose increase. This effect was more pronounced for patients with stronger glucose responses to meals or higher insulin sensitivity. Consequently, reward after 300,000 steps was useful for comparing early learning behavior during hyperparameter tuning, but it was not always fully representative of final converged performance. The learning curves with average total episode reward can be found in Appendix C.

Parameter	Value
reward_type	clarke_risk
learning_rate	4.00×10^{-4}
n_steps	1920
batch_size	160
n_epochs	10
gamma	0.99
gae_lambda	0.90
clip_range	0.20
ent_coef	5.00×10^{-3}
vf_coef	0.70
max_grad_norm	0.80
net_arch	[128, 128, 64]

Table 3: Best hyperparameter configuration across patients from Optuna parameter search for HCL model.

Parameter	Value
reward_type	clarke_risk
learning_rate	2.00×10^{-4}
n_steps	1440
batch_size	80
n_epochs	15
gamma	0.99
gae_lambda	0.95
clip_range	0.25
ent_coef	5.00×10^{-3}
vf_coef	0.50
max_grad_norm	0.70
net_arch	[128, 128, 64]

Table 4: Best hyperparameter configuration across patients from Optuna parameter search for FCL model.

5.2.4 Policy Distillation Training

The symbolic regression target was the teacher's mapped insulin dose $u_t \in [0, 5]$ rather than the raw PPO action $a_t \in [-1, 1]$. This choice was important because the PPO action was transformed to insulin delivery through an exponential mapping. As a result, distances in the normalized action space do not correspond linearly to differences in delivered insulin: small errors in a_t can lead to substantially larger differences in u_t, particularly near the upper end of the action range. Training the symbolic student directly on u_t therefore aligned the regression objective with the physiological control input applied to the simulated patient and made the resulting expression easier to interpret as an insulin-dosing rule. During rollout, the predicted symbolic insulin output was clipped to the valid range and transformed back to the normalized environment action before being passed to the simulator.

Since insulin dosing is a safety-critical control problem, the symbolic search space was deliberately restricted. Preliminary experiments with more expressive operators showed that division, logarithms, square roots, and exponential functions could produce undefined or numerically unstable expressions when valid state variables were zero or near zero. These operators were hence

removed from the symbolic search space fed into PySR. The final PySR search space candidates was restricted to addition, subtraction, multiplication, optional threshold operators, and at most one level of squaring. In addition expression depth and nesting were constrained and parsimony set to favor stable and low complexity dosing rules without abrupt action changes. Threshold operators were evaluated because they can represent clinically interpretable range-dependent dosing rules.

We tested the following combinations of the configurations:

Table 5: PySR configurations used for symbolic policy distillation.

Configuration	Binary operators	Unary operators	Max. size
`linear_10`	$+, -, \times$	None	10
`square_10`	$+, -, \times$	square	10
`square_threshold_10`	$+, -, \times, <, >$	square	10
`square_threshold_15`	$+, -, \times, <, >$	square	15
`square_threshold_20`	$+, -, \times, <, >$	square	20

The configurations were chosen to represent increasing symbolic expressivity. The linear configuration served as the simplest baseline. The square configuration allowed smooth nonlinear responses to the input variables, while the threshold configurations allowed piecewise decision rules. The maximum expression size was varied to test whether more complex symbolic expressions improved fidelity and clinical performance.

For each patient and PySR configuration, the distillation process was repeated five times to account for the stochastic nature of symbolic regression. Each distilled policy was evaluated using the clinical metrics described in Section 5.2.1.3, and the resulting symbolic expressions were stored for qualitative inspection.

Because repeated GM-DAgger/SPID training was computationally expensive, the symbolic regression configuration was selected based on adult-001. The selected configurations were then fixed and reused for all remaining patients, ensuring a consistent evaluation protocol across the cohort. This made the full evaluation feasible, but is also a methodological limitation, since the search space was not independently optimized for each patient.

Configuration selection was based on the best-performing run among the five repetitions, while also considering consistency across runs and safety-related metrics. In the HCL setting, threshold-based configurations generally performed best. The configuration `square_threshold_15` achieved a reward close to the best alternatives while giving the lowest CFR by a small margin, and was therefore selected for the remaining HCL distillations.

In the FCL setting, `linear_10` performed consistently well across repetitions and achieved the strongest overall performance. This configuration was therefore selected for the remaining FCL distillations.

5.3 Results

For each of the tasks of FCL and HCL control, we evaluate the models' performance in terms of the evaluation metrics as described in Section 5.2.1.3. The evaluation is performed on the patient-specific trained models, and compared against the performance of the benchmark control algorithms as presented in Section 5.2.1 on a individual patient level. For the HCL case, we compare against the open-loop control algorithm BBH, and in the FCL scenario, we evaluate the PPO and symbolic policy (SP) against a FCL PPO-controller.

We summarize the training results and achieved performance of the PPO models, and assess how the distilled SPs compare against the teacher models. For the SPs we present the derived expressions along with an interpretation of the policy. Lastly, we demonstrate how PPO and SP perform over a prolonged three-day episode, where meals occur with larger variability in both size and timing to assess the robustness of our policies.

5.3.1 Hybrid Closed-Loop Control

A patient-specific PPO model was tuned and trained as described in Section 5.2.3. Across the ten virtual patients, the resulting models generally converged and achieved moderate to strong performance. However, learning dynamics varied strongly between patients. Several models converged steadily and reached strong final performance, with TIR above 80% and CFR below 15%. In contrast, other patients exhibited slower convergence and plateaued at lower performance levels, with CFR values between 25% and 40%, indicating that many patients did not make it to the end of the episode.

Figure 9 shows representative evaluation curves for adult-001 and adult-006, corresponding to a high-performing and lower-performing model, respectively. The curves report mean TIR, CFR, and episode reward at evaluation intervals of 100,000 training steps. Training curves for all models are provided in Appendix C.1.

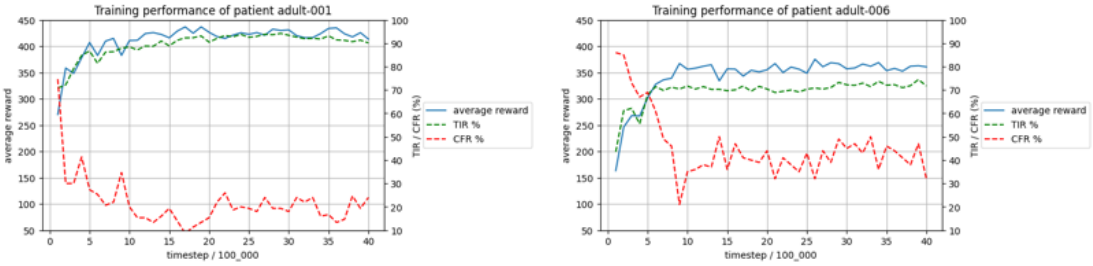


Figure 9: Test evaluations at every 100 000 training steps, reporting mean TIR, CFR and reward across a 100 simulated episodes for adult-001 and adult-006

Overall, the trained PPO controllers achieved performance comparable to the BBH benchmark across all patients. The final performance of the PPO models in the HCL setting is shown in Table 6. The BBH controller showed stable performance across patients, with TIR generally in the range of 70-80%. In most cases, the PPO controllers achieved similar or higher TIR, ranging from 72-98%. Moreover, BBH showed greater episode-to-episode variability, suggesting higher sensitivity to meal noise, whereas PPO generally produced more consistent outcomes across episodes.

Despite this overall improvement, performance differences were strongly patient-dependent. Poor performance was often observed for the same patients across both controllers, indicating that some virtual patients are inherently more difficult to stabilize. For these more sensitive patients, the PPO controllers generally achieved higher TIR but also a higher CFR than BBH. This suggests that the learned policies spend more time in range, but at the cost of more frequent early episode terminations.

A likely explanation is that BBH tends to be more conservative, spending a larger fraction of time above range, often 20-30%. This reduces the risk of hypoglycemic critical failures, but also limits the attainable TIR. In contrast, the PPO controllers appear to control glucose more aggressively, thereby increasing time in range while also increasing the risk of hypoglycemia-related failures in more sensitive patients.

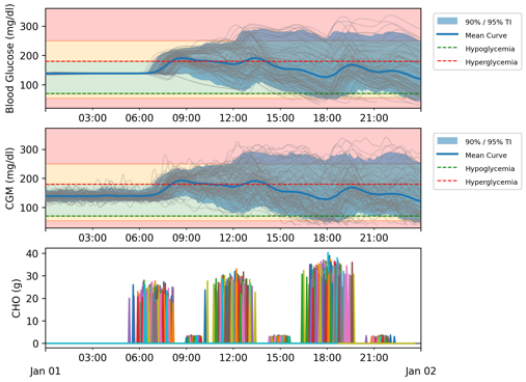
Patient	Model	TIR (%)	TBR I (%)	TAR I (%)	CFR (%)	Reward
adult-001	HCL-SP	94.2 ±4.8	2.1 ±3.0	3.7 ±3.8	7.7	439.9 ±47.3
	HCL-PPO	93.0 ±5.9	3.2 ±4.3	3.8 ±3.6	19.3	426.3 ±58.9
	BBH	74.6 ±15.4	1.8 ±3.5	23.6 ±15.7	15.7	383.3 ±53.5
adult-002	HCL-SP	85.8 ±5.6	3.4 ±3.2	10.8 ±4.1	5.7	425.2 ±27.1
	HCL-PPO	86.8 ±5.1	9.3 ±4.5	3.9 ±2.6	35.7	405.1 ±51.9
	BBH	77.7 ±15.6	1.6 ±3.7	20.7 ±15.3	5.3	402.4 ±46.8
adult-003	HCL-SP	85.9 ±6.7	3.0 ±4.0	11.1 ±4.3	8.3	417.8 ±54.4
	HCL-PPO	90.1 ±5.8	2.5 ±3.8	7.3 ±3.9	11.7	416.8 ±80.1
	BBH	73.2 ±17.4	2.5 ±4.3	24.3 ±16.7	29.3	348.2 ±83.0
adult-004	HCL-SP	73.2 ±9.4	18.4 ±7.5	8.3 ±3.2	19.7	371.7 ±63.1
	HCL-PPO	80.8 ±8.2	8.0 ±6.3	11.2 ±3.2	13.0	397.4 ±55.2
	BBH	70.1 ±16.5	3.0 ±5.2	26.9 ±16.1	12.7	364.5 ±64.1
adult-005	HCL-SP	75.1 ±7.2	1.3 ±2.3	23.6 ±6.7	5.3	385.9 ±49.6
	HCL-PPO	79.3 ±8.0	2.3 ±3.0	18.4 ±7.2	13.7	379.5 ±77.5
	BBH	74.9 ±18.0	2.3 ±4.1	22.8 ±17.4	22.3	370.9 ±73.5
adult-006	HCL-SP	73.9 ±6.0	5.2 ±3.2	20.9 ±4.0	45.0	361.9 ±70.0
	HCL-PPO	72.7 ±7.2	5.6 ±3.8	21.6 ±5.3	40.3	367.8 ±54.8
	BBH	74.5 ±15.2	2.2 ±3.4	23.3 ±14.4	36.0	345.1 ±80.7
adult-007	HCL-SP	79.1 ±6.4	7.9 ±4.7	13.0 ±4.3	28.0	373.2 ±72.8
	HCL-PPO	75.2 ±9.1	10.5 ±6.1	14.3 ±4.3	33.7	367.4 ±65.9
	BBH	75.0 ±16.2	3.6 ±5.7	21.4 ±14.7	32.0	358.4 ±77.8
adult-008	HCL-SP	99.6 ±1.3	0.2 ±1.1	0.2 ±0.8	0.7	466.8 ±16.8
	HCL-PPO	97.9 ±3.1	1.1 ±2.6	1.0 ±1.6	4.7	454.1 ±42.8
	BBH	80.3 ±18.6	2.4 ±5.2	17.4 ±17.6	12.0	401.5 ±57.8
adult-009	HCL-SP	93.7 ±4.9	0.9 ±3.0	5.4 ±3.6	3.0	443.9 ±25.4
	HCL-PPO	92.6 ±6.3	3.3 ±5.2	4.1 ±3.7	15.7	420.3 ±73.9
	BBH	74.3 ±17.6	1.3 ±2.5	24.4 ±18.0	14.7	378.5 ±60.2
adult-010	HCL-SP	81.0 ±5.8	4.8 ±3.3	14.2 ±3.9	31.3	382.7 ±67.8
	HCL-PPO	83.4 ±5.2	5.3 ±3.8	11.3 ±3.4	26.7	395.4 ±65.4
	BBH	70.5 ±16.2	1.3 ±3.0	28.1 ±16.2	11.0	367.7 ±59.1

Table 6: Overview of HCL-controller performance compared to the BBH baseline across patient 001-010. Values are measured as the mean across 300 simulated episodes with standard deviation

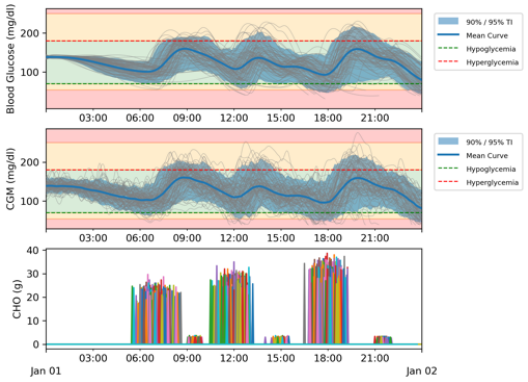
From each trained HCL-PPO model, we distilled a patient-specific symbolic policy as described in Section 5.2.4. For each patient, the distillation procedure was repeated five times. Each resulting symbolic policy was evaluated over 100 episodes, and the expression with the highest average Clarke Risk reward was selected as the final policy. The final symbolic policies, together with their policy reward and TIR, are shown in Table 7.

Overall, the distilled symbolic policies achieved strong performance and closely matched their teacher PPO models in terms of TIR, as shown in Table 6. In terms of CFR, the symbolic policies often outperform both PPO and BBH. This tendency is mostly prominent for patients, where the PPO teacher itself achieves a high average episode reward. In contrast, for patients where PPO scores low average episode reward, which often corresponded to more sensitive patients, the distilled policies tended to reproduce the same limitations as the teacher. This was especially apparent for patients such as 006 and 007, where both the teacher and symbolic policies achieved lower CFR-related performance.

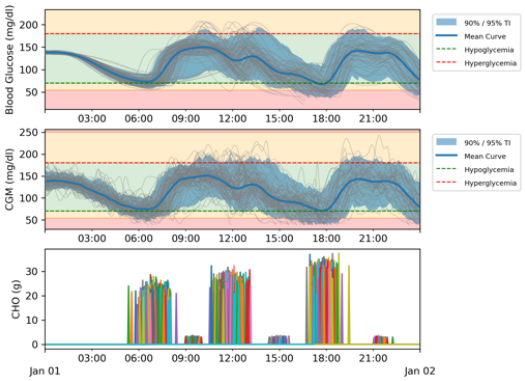
An example of the simulated episodes for adult-001 using the trained HCL-PPO, HCL-SP and BBH baseline can be seen in Figure 10



(a) BBH



(b) PPO



(c) SP

Figure 10: 100 simulated trajectories for BBH, PPO and SP over the period of 24 hours

Model	Patient	Policy	TIR(%)	Reward
HCL-SP	adult-001	$\begin{cases} 1.00 & \text{for } x_0 > x_2 + 0.286 \\ 0.0 & \text{otherwise} \end{cases}$	94.2	439.9
	adult-002	$\begin{cases} 1.00 & \text{for } x_0 > x_2 + 0.300 \\ 0.0 & \text{otherwise} \end{cases}$	85.8	425.2
	adult-003	$\begin{cases} 1.00 & \text{for } x_0 > x_2 + 0.312 \\ 0.0 & \text{otherwise} \end{cases}$	85.9	417.8
	adult-004	$x_0 - x_2 \cdot 2.70 - 0.108$	73.2	371.7
	adult-005	$\begin{cases} 1.00 & \text{for } x_0 > x_1 + x_2 + 0.231 \\ 0.0 & \text{otherwise} \end{cases}$	75.1	385.9
	adult-006	$x_0^2 - x_2$	73.9	361.9
	adult-007	$x_0 \cdot 0.108$	79.1	373.2
	adult-008	$x_0^2 - x_2$	99.6	466.8
	adult-009	$\begin{cases} 1.00 & \text{for } x_0 > x_2 + 0.302 \\ 0.0 & \text{otherwise} \end{cases}$	93.7	443.9
	adult-010	$\begin{cases} 1.00 & \text{for } x_0 > x_2^2 + 0.417 \\ 0.0 & \text{otherwise} \end{cases}$	81.0	382.7

Table 7: Overview of HCL symbolic policy expressions distilled from patient HCL-PPO models, alongside the policies' average TIR and average episode reward

5.3.1.1 Symbolic Policies As shown in Table 7, the derived symbolic policies take the form of relatively simple expressions, yet still achieve high performance using only a small number of variables and operators. With the exception of adult-005, all policies depend exclusively on the variables x_0 and x_2, corresponding to the measured CGM value and IOB, respectively. The variable x_1, which appears only in the expression for adult-005, represents the time since the last meal.

Because the symbolic policies rely primarily on two variables, their behavior can be visualized as two-dimensional heatmaps, as illustrated in Figure 11.

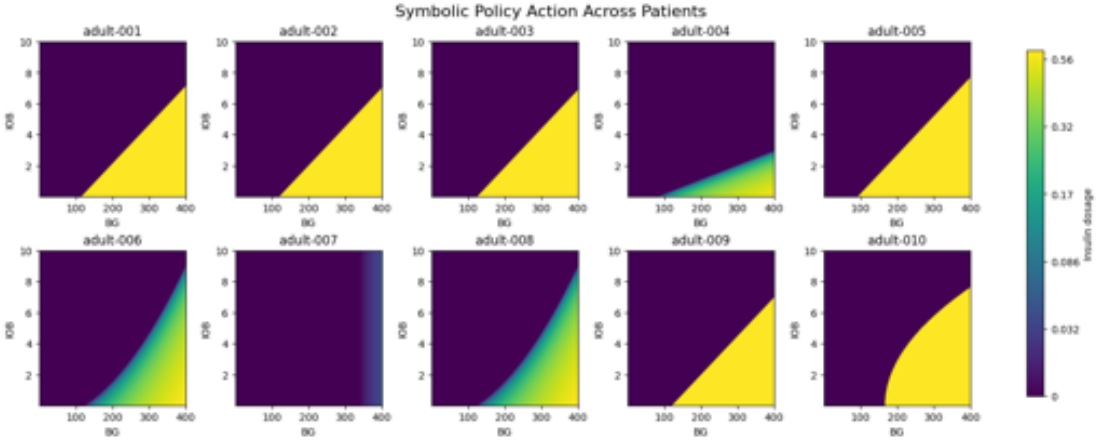


Figure 11: Heatmaps of symbolic policy actions as a function of measured BG and IOB. The color scale is log-transformed to make both basal-level doses and larger correction doses visible within the same plot. Although the heatmaps cover a broad range of BG and IOB values, the policies are unlikely to visit the full displayed state space during evaluation. In particular, combinations of high BG and high IOB are rare, since elevated IOB typically results from recent insulin delivery. For adult-005, the heatmap is evaluated under the assumption that a meal has just been consumed, such that $x_1 = x_{\text{time since meal}} = 0$.

The extracted symbolic policies show strong similarity across patients. Although parameter tuning allowed for the discovery of more complex expressions, these did not lead to improved performance. We therefore selected the simplest policies among those achieving the highest evaluation performance.

Across several patients, the symbolic expressions describe an insulin-dosing strategy based on a dynamic threshold. In these policies, insulin is administered when the measured BG, represented by the CGM value, exceeds the IOB term by a patient-specific constant. When this condition is satisfied, the policy delivers a fixed insulin dose of 1 U/min. These policies do not practice maintaining a constant basal level by continuous dosing, but administers a moderate insulin dosis when needed. As shown in Figure 11, this results in a straightforward linear decision boundary, with highly similar policies for adult-001, adult-002, adult-003, adult-005, and adult-009.

Except for adult-005, none of these policies use meal-related information. In this sense, the extracted symbolic policies largely behave as FCL controllers, relying only on the current glucose state and IOB. For adult-005, the dynamic threshold additionally depends on the time since the last meal. This means that insulin is administered when blood glucose exceeds the IOB-dependent threshold and sufficient time has passed since the previous meal.

The adult-005 policy may therefore indicate a brittle control strategy. Rather than reacting only to elevated glucose levels or explicit meal intake, the policy may be dosing partly in anticipation of future glucose increases. This could suggest that the underlying PPO model has learned to exploit the relatively regular spacing between meals in the training environment, which may be necessary for its observed performance.

As illustrated in Appendix C.1, several of the models that were harder to train, such as adult-006, adult-007, and adult-010, also produced symbolic policies with slightly different structure. These policies generally exhibit softer dosing behavior and more nonlinear relationships. This may either reflect suboptimal teacher policies, resulting in symbolic policies that differ from the stronger models, or indicate that these patients require more sensitive and patient-specific control strategies. As discussed above, these patients also appear more difficult to stabilize.

5.3.2 Closed Loop

FCL PPO models and symbolic policies were trained and distilled using the same procedure as described for HCL. With the addition of previous actions added to the state-space, and no meal info, we observed that most patient models across achieved high TIR scores within the range of 75% to 99% and CFR below 15%, with the exception of adult-004 and adult-006. The model trained for adult-004 did not converge, and 99% of patient do not survive a full episode. For adult-006 we only achieved moderate performance, with more than 30% of the virtual patients not surviving an episode. Final scores are displayed in Table 8, and the models corresponding evaluations curves during training can be seen in Appendix C.2.

In terms of TIR, the performance of the PPO models is comparable to that of PID, however, in terms of CFR, the PPO models often strongly outperform PID. PID often resuls in CFR values in the range of upwards 20-40%, whereas PPO often CFR lies in the range of 5-15% (with the exception of adult-004 and adult-006).

Patient	Model	TIR	TBR I	TAR I	CFR	Reward
adult-001	CL-SP	97.2 ±4.0	2.3 ±3.8	0.4 ±1.2	12.3	446.4 ±43.6
	CL-PPO	88.8 ±6.5	3.3 ±4.5	8.0 ±5.1	15.0	417.0 ±63.0
	PID	94.8 ±6.7	4.5 ±3.9	0.7 ±5.9	23.3	421.0 ±72.9
adult-002	CL-SP	89.7 ±6.1	2.8 ±3.4	7.5 ±4.9	3.3	434.4 ±22.7
	CL-PPO	88.7 ±5.3	4.0 ±3.5	7.3 ±4.2	7.0	426.2 ±50.2
	PID	91.4 ±5.8	7.8 ±4.8	0.7 ±4.5	22.7	417.8 ±53.4
adult-003	CL-SP	76.3 ±8.9	8.2 ±4.4	15.5 ±6.5	16.7	369.7 ±88.8
	CL-PPO	92.8 ±5.4	1.8 ±2.8	5.4 ±4.0	6.3	435.2 ±50.7
	PID	86.8 ±5.6	12.2 ±3.1	1.0 ±6.6	52.7	268.3 ±161.2
adult-004	CL-SP	69.1 ±4.9	1.1 ±1.9	29.8 ±4.9	2.0	381.3 ±14.6
	CL-PPO	31.1 ±0.9	10.7 ±8.2	58.2 ±7.7	99.0	128.9 ±56.4
	PID	77.3 ±8.4	19.2 ±7.6	3.5 ±5.4	18.0	370.0 ±102.2
adult-005	CL-SP	94.2 ±6.2	2.1 ±4.4	3.7 ±3.5	2.3	437.6 ±42.7
	CL-PPO	89.1 ±5.8	2.5 ±4.7	8.5 ±4.4	11.0	406.6 ±100.2
	PID	86.9 ±8.5	12.0 ±7.3	1.1 ±6.3	31.0	355.8 ±131.2
adult-006	CL-SP	67.9 ±6.3	1.9 ±3.5	30.2 ±5.2	7.3	375.1 ±33.1
	CL-PPO	74.1 ±5.8	5.5 ±3.5	20.4 ±4.4	32.7	372.6 ±62.7
	PID	74.7 ±8.6	19.6 ±6.1	5.7 ±7.6	69.7	220.4 ±142.1
adult-007	CL-SP	85.9 ±7.1	9.5 ±6.0	4.6 ±4.4	20.3	381.2 ±103.6
	CL-PPO	86.4 ±5.5	3.2 ±3.4	10.4 ±3.9	4.3	422.4 ±43.3
	PID	77.4 ±7.9	20.2 ±6.6	2.4 ±3.7	42.0	316.5 ±133.5
adult-008	CL-SP	99.1 ±3.2	0.9 ±3.2	0.0 ±0.3	1.0	467.4 ±12.7
	CL-PPO	98.5 ±3.6	1.4 ±3.5	0.1 ±0.7	11.3	439.8 ±82.0
	PID	97.9 ±7.1	1.6 ±5.1	0.4 ±5.0	7.3	451.7 ±53.6
adult-009	CL-SP	92.9 ±5.3	6.8 ±5.1	0.3 ±1.0	26.3	413.1 ±73.1
	CL-PPO	95.2 ±5.5	2.1 ±4.1	2.7 ±3.3	9.7	435.2 ±57.6
	PID	93.8 ±7.3	5.1 ±3.3	1.1 ±7.3	31.7	391.7 ±105.7
adult-010	CL-SP	95.5 ±5.5	2.9 ±4.6	1.6 ±2.7	9.3	431.4 ±65.9
	CL-PPO	85.6 ±6.2	3.3 ±4.0	11.1 ±5.1	12.3	414.2 ±52.8
	PID	87.3 ±6.7	11.8 ±4.1	1.0 ±7.0	29.3	372.8 ±112.1

Table 8: Overview of FCL-controller performance compared to the PID baseline across patient 001-010. Values are measured as the mean across 300 simulated episodes with standard deviation

The distilled SPs show good performance across most patients, with some variance. With the exception of adult-004 and adult-006, all policies have strong performance with TIR measures in the range 76-99%. CFR is more variable between patients, with some patients achieving substantially low values of 1-3% and others upwards 20-26%. Unlike in the case of the distilled

HCL-SP, we sometimes observe the student having noticeable higher CFR than it's teacher with a corresponding reduction in average episode reward, potentially indicating that the SP is not fully capable of replicating teacher performance.

Model	Patient	Policy	TIR (%)	Reward
CL-SP	adult-001	$x_0 \cdot 0.155$	97.2	446.38
	adult-002	$x_0^2 \, (0.528 - x_1)$	89.7	434.4
	adult-003	$x_0^2 x_2 x_6$	76.3	369.7
	adult-004	$x_0^2 \cdot 0.133 - x_0 x_1 \cdot 0.133$	69.1	381.3
	adult-005	$x_0 \cdot 0.177$	94.2	437.6
	adult-006	$x_0 \cdot 0.118$	67.9	375.1
	adult-007	$x_5 \cdot 0.0944 x_6$	85.9	381.2
	adult-008	$x_5 \cdot 0.0953 \, (-x_4 + x_5)$	99.1	467.4
	adult-009	$x_0^2 - x_0 x_1$	92.9	413.1
	adult-010	$(x_2 + x_6) \, 0.115 x_3$	95.5	431.4

Table 9: Symbolic Policies based on distillation of FCL PPO controller

5.3.2.1 Symbolic Policies The symbolic policies for FCL control were derived using only addition, multiplication and subtraction for the distillation (Section 5.2.4). Whilst the choice of operators is reduced, the derived expressions are quite varied, and make use of more variables than observed in the HCL scenario. Some policies, such as the policies for adult-001, adult-005 and adult-006 rely on very simple expressions, where insulin is dosed proportionally to BG levels, and yet achieve strong performance.

Adult-002, adult-004, and adult-009 obtain structurally similar symbolic policies of the form $a \, x_0^2 - b \, x_0 \, x_1$. In these policies, the insulin dose increases non-linearly with the current CGM level through the quadratic term x_0^2. This means that higher glucose values lead to disproportionately larger insulin recommendations. The second term, $-b \, x_0 \, x_1$, acts as a corrective IOB component: when IOB is high, the recommended dose is reduced, and this reduction is scaled by the current glucose level. Thus, the policy can be interpreted as dosing aggressively under hyperglycemia while limiting additional insulin when residual active insulin is already present.

For adult-007, adult-008 and adult-010, the derived symbolic policies do not rely on any environment variables at all, but solely on previous actions taken; receiving no feed-back from the system. These policies appear to be balancing insulin dosing by adjusting levels *recursively*; if low amounts have been administered, the expressions generally encourage larger dosing in the next step, and if high amounts have been given, they will tend to reduce (we make note the action variables x_2 - x_6 are in fact normalized to the ranges (-1, 1) as described in Section 5.2.2, and so small insulin dosages are negative, whilst larger dosages are given at positive values). Even so, some of these policies manage to achieve high performance, often exceeding the PPO model.

The high performance of these action-history policies may indicate that they exploit regularities in the simulated environment. Since the policies do not condition directly on current

glucose or IOB, their performance is unlikely to reflect robust feedback control. Instead, they may approximate a dosing rhythm that is sufficient under the relatively regular meal structure and fixed episode dynamics of the evaluation setting. Such policies may therefore be brittle: they can perform well under familiar meal timing and episode conditions, but may be less robust if meal timing, meal size, or patient dynamics deviate from those encountered during training. Whether this behavior originates from the symbolic distillation procedure itself or reflects similar tendencies in the teacher PPO policy remains unclear.

The mismatch in reward and CFR between PPO teacher and SP student as observed for some patients, such as adult-003, adult-007, and adult-009, may suggests the same. In these cases, the symbolic policy may achieve acceptable average reward or TIR while still producing unstable behavior, reflected in elevated CFR. This suggests that some of the distilled expressions may be too simple to capture the full feedback structure of the teacher policy, particularly for patients that require more sensitive control.

5.3.3 Robustness of symbolic policies

To assess the robustness of both the trained PPO models and the distilled symbolic policies, we evaluate the controllers under a more challenging simulation setting. In addition to the standard one-day evaluation scenario, using the same level of meal variability as during training, we run a stress-test experiment consisting of three-day episodes with increased variability in both meal timing and meal size. The experiment is conducted for both the HCL and FCL settings, using 300 simulated episodes for each virtual patient. The results are shown in Figure 12.

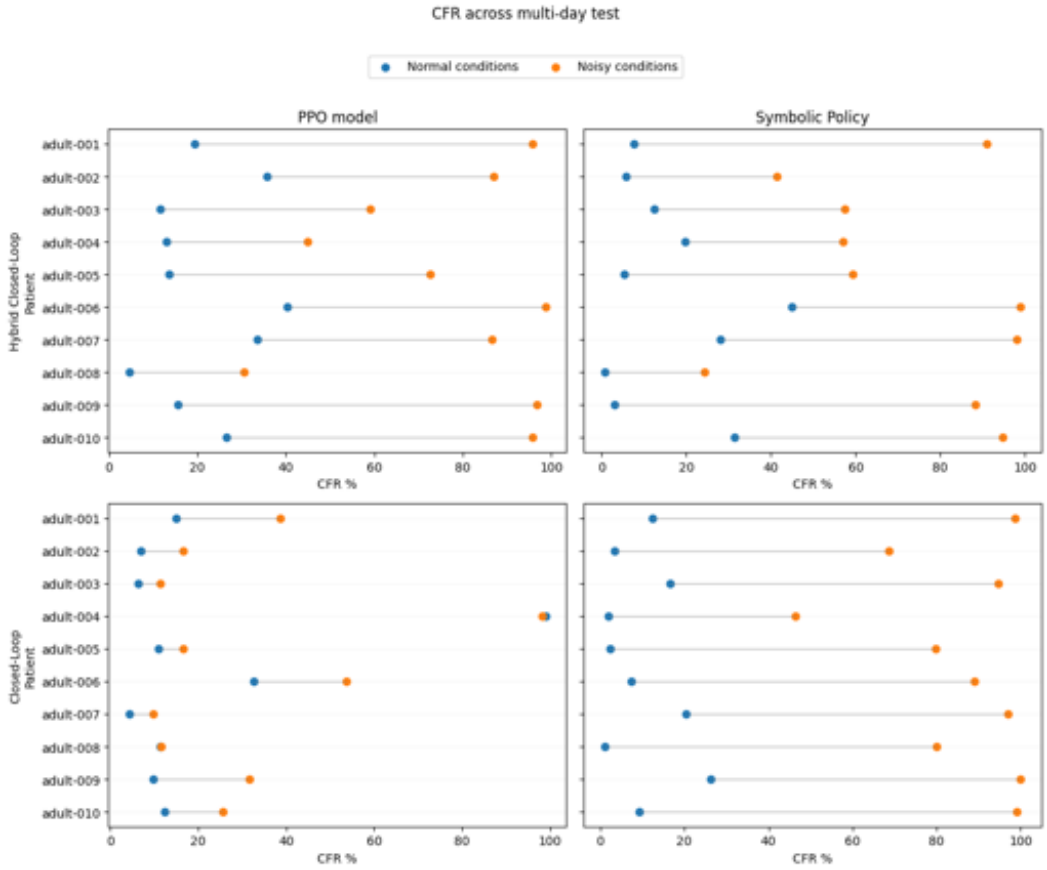


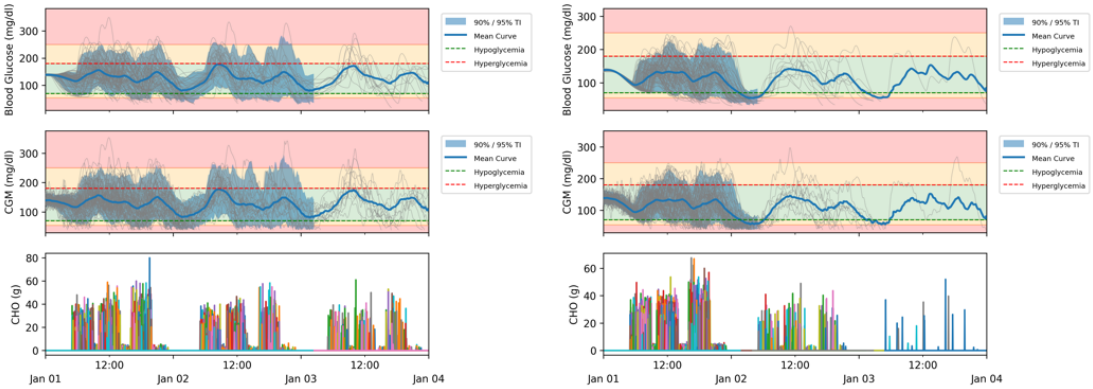
Figure 12: Comparison of average CFR between two experiments conducted across 300 simulations. The normal condition consists of one-day episodes with the standard level of meal variability used during training, shown by blue dots. The noisy condition consists of three-day episodes with increased meal variability, shown by orange dots. The left plot shows the difference for the PPO models for each patient, while the right plot shows the corresponding difference for the distilled symbolic policies.

Robustness is evaluated in terms of CFR. This metric is used because TIR is computed only for completed episodes, and is therefore less informative when many episodes terminate early. Similarly, total reward is not directly comparable between the two experiments, since the episode lengths differ. The CFR therefore provides a direct measure of whether the controller remains safe and stable under longer and more variable conditions.

The stress-test experiment reveals clear differences between the control methods. For the neural PPO controllers, the FCL models generally retain their performance better than the HCL models, even though the CFR increases under noisy conditions. This is somewhat surprising, since access to meal information would normally be expected to make the control task easier. However, the result suggests that the HCL policies may be more sensitive to changes in the meal distribution, particularly when the timing and size of meals deviate from the conditions encountered during training. In contrast, the FCL models appear to rely more directly on the observed glucose dynamics, which may make them less dependent on the exact meal-information structure.

The symbolic policies are generally less robust than their PPO teachers, although the degree of degradation differs substantially between the HCL and FCL settings. For the HCL-SP models, the symbolic policies exhibit a level of performance degradation similar to that of their teachers. This indicates that, although performance decreases under the noisy three-day setting, the

symbolic policies remain relatively faithful to the teacher models in terms of robustness. In contrast, the FCL-SP models show a much larger degradation, with the CFR increasing by approximately 80% in several cases. As a result, most virtual patients fail to survive the full three-day evaluation period. An example of this behavior for adult-001 is shown in Figure 13.



(a) 300 simulated episodes of the adult-001 PPO model.

(b) 300 simulated episodes of the adult-001 symbolic policy.

Figure 13: Example trajectories for adult-001 over a three-day period with increased meal variability.

The mismatch between the HCL and FCL symbolic policies highlights an important caveat of symbolic policy distillation. In this work, symbolic policies were selected based on both performance and expression complexity. In the FCL case, particularly simple expressions were selected because they achieved strong reward-based performance under the standard evaluation setting. However, the stress-test results suggest that these expressions may underfit the teacher policy. Rather than faithfully capturing the teacher's decision-making behavior, the symbolic policy may exploit specific dynamics of the simulation environment that are sufficient under the standard one-day setting, but fail under distribution shift.

This issue is closely related to the objective used during distillation. The GM-DAgger loss allows the symbolic policy to deviate from the teacher when such deviations improve performance. This is useful when the symbolic policy can discover a simpler policy that preserves or improves control performance. However, it also introduces a potential failure mode: if the symbolic function class is too restricted, the loss may be minimized primarily through the performance component rather than through high fidelity to the teacher. In such cases, the resulting policy may achieve strong performance in the training or validation environment without truly reproducing the teacher policy. The policy may therefore appear successful, but generalize poorly when the environment dynamics or meal distributions are altered.

The results for the HCL symbolic policies suggest the opposite case. Although both the HCL PPO models and the corresponding symbolic policies degrade under the noisy three-day setting, the symbolic policies degrade in a manner similar to their teachers. This indicates that the higher complexity of the HCL symbolic policies may have allowed them to preserve stronger fidelity to the teacher. This interpretation is further supported by the qualitative structure of the HCL symbolic expressions, which were generally more interpretable and clinically meaningful than the FCL symbolic policies. In contrast, several of the FCL symbolic expressions relied on less meaningful variable combinations, suggesting that their strong standard-setting performance may not reflect a robust or clinically sensible control strategy.

Overall, the robustness experiment shows that symbolic policies can generalize beyond the conditions under which they were selected, but that this depends strongly on the quality and complexity of the distilled expression. Very simple symbolic policies may be attractive because

they are interpretable and easy to inspect, but they may also underfit the teacher or exploit simulator-specific dynamics. Conversely, slightly more complex symbolic policies may better preserve teacher fidelity and robustness, even if they are less compact. These results emphasize that symbolic policy selection should not be based on reward and simplicity alone, but should also consider fidelity, robustness under distribution shift, and the clinical plausibility of the derived expression.

More broadly, this experiment also illustrates a limitation of training and evaluating DRL controllers in simulated environments. Even when a controller achieves strong performance in simulation, it may do so by exploiting simplified or biased environment dynamics rather than by learning a generally safe insulin-dosing strategy. Symbolic distillation can make such behavior easier to inspect, but it may also expose non-sensical policies that nevertheless perform well under limited evaluation conditions. This reinforces the need for stress testing, robustness analysis, and careful interpretation when evaluating both neural and symbolic glucose-control policies.

5.3.4 An Experimental Distillation of a Three-day Episode

As a final exploratory test, we distilled the FCL-PPO teacher for a single patient, adult-001, under the more challenging three-day evaluation setting. The purpose of this experiment was to assess whether symbolic policies extracted from longer and noisier roll-outs would differ substantially from those obtained in the standard one-day setting. For this experiment, roll-outs were collected over three days with increased variability in meal timing and meal size. Importantly, the PPO teacher was not trained under these conditions, making this test a form of out-of-distribution evaluation.

The results show that symbolic policies obtained with the `square_threshold_15` configuration perform well in this extended setting. In several cases, the distilled policies strongly outperform the teacher with respect to CFR, indicating substantially improved safety under the three-day evaluation horizon. The best-performing symbolic student achieves a CFR of only 0.7%, compared with considerably higher failure rates for several of the simpler `linear_10` policies. In contrast, the `linear_10` configuration appears too restrictive to generalize reliably to extended episodes, with multiple policies producing CFR values above 50%.

The extracted `square_threshold_15` policies are also structurally consistent with the threshold-based policies observed in the HCL setting, as shown in Table 10. A representative policy for adult-001 is given by

$$\begin{cases} 1.00 & \text{for } x_0 > x_2 + 0.286, \\ 0.0 & \text{otherwise.} \end{cases} \tag{17}$$

This policy follows a simple threshold logic: insulin is administered only when the current BG state variable exceeds IOB term by a fixed margin. Similar threshold structures are observed across the highest-performing policies in Table 10, where insulin delivery depends on comparisons between glucose and IOB variables.

Although these policies achieve low CFR, this comes at the cost of reduced TIR. Compared with the previously extracted one-day policies, the three-day `square_threshold_15` policies tend to spend more time above range, with TAR often increasing to approximately 20–30%. This suggests a more conservative long-horizon strategy: rather than aggressively minimizing hyperglycemia, the symbolic policies appear to tolerate more time above range in order to reduce the risk of hypoglycemia and early episode termination. This is reflected in the higher dosing thresholds, where insulin is only delivered once the glucose-related state exceeds the relevant comparison term by a larger margin.

These results highlight the importance of the environment and reward design in shaping the final symbolic policies, and how strongly the performance term of the GM-DAgger-loss affects the policy. Over a multi-day horizon, survival becomes increasingly important for achieving high return, since early termination removes a larger amount of potential future reward. Consequently, the distilled symbolic policies may learn to exploit aspects of the evaluation design by prioritizing long-term survival over tighter glucose regulation. This should be interpreted as both a strength and a limitation: the symbolic policies reveal the strategy encouraged by the reward structure, but this strategy may not necessarily transfer unchanged to even longer rollouts or to clinical settings with different safety requirements.

The experiment also emphasizes the value of well-defined state variables. Across several extracted policies, variables related to glucose level and IOB appear as central components of the control logic. This supports the interpretation that the symbolic regression procedure is able to identify clinically meaningful structure when relevant variables are made available. In particular, the repeated emergence of threshold-based rules involving glucose and insulin-related terms suggests that compact symbolic policies can capture meaningful control mechanisms even in extended and noisier evaluation settings.

Policy Setting	TIR (%)	Reward	CFR(%)	equation
square_threshold_15	69.8	858.6	8.7	$\begin{cases} 1.00 & \text{for } x_0 - x_1 > 0.408 \\ 0.0 & \text{otherwise} \end{cases}$
`square_threshold_15`	63.0	887.8	4.3	$\begin{cases} 1.00 & \text{for } x_0 - x_1 > 0.438 \\ 0.0 & \text{otherwise} \end{cases}$
`square_threshold_15`	71.6	825.9	7.3	$\begin{cases} 1.00 & \text{for } x_0 - x_1 > 0.398 \\ 0.0 & \text{otherwise} \end{cases}$
`square_threshold_15`	72.2	827.3	10.0	$\begin{cases} 1.00 & \text{for } x_0 > x_1 - -0.388 \\ 0.0 & \text{otherwise} \end{cases}$
`square_threshold_15`	70.4	1032.3	0.7	$\begin{cases} 1.00 & \text{for } x_1 + 0.347 < x_0 - x_1 \\ 0.0 & \text{otherwise} \end{cases}$
`linear_10`	82.3	455.4	56.3	$x_0 \, (x_0 - 0.197)$
`linear_10`	78.6	427.2	55.7	$x_3 \, (0.216 - x_0)$
`linear_10`	77.4	696.4	22.7	$x_0 x_0 \, (x_0 - x_1)$
`linear_10`	80.5	375.9	61.7	$x_0 \, (x_0 - x_1)$
`linear_10`	79.5	378.1	60.3	$x_0 \, (x_0 - x_1)$

Table 10: Performance metrics and symbolic policies for ten adult-001 students distilled from the FCL-PPO teacher over a three-day evaluation horizon.

The policy with the highest reward is shown in Figure 14. This policy was obtained using the `square_threshold_15` configuration and selected among the five threshold-based distillation runs based on high reward and low CFR.

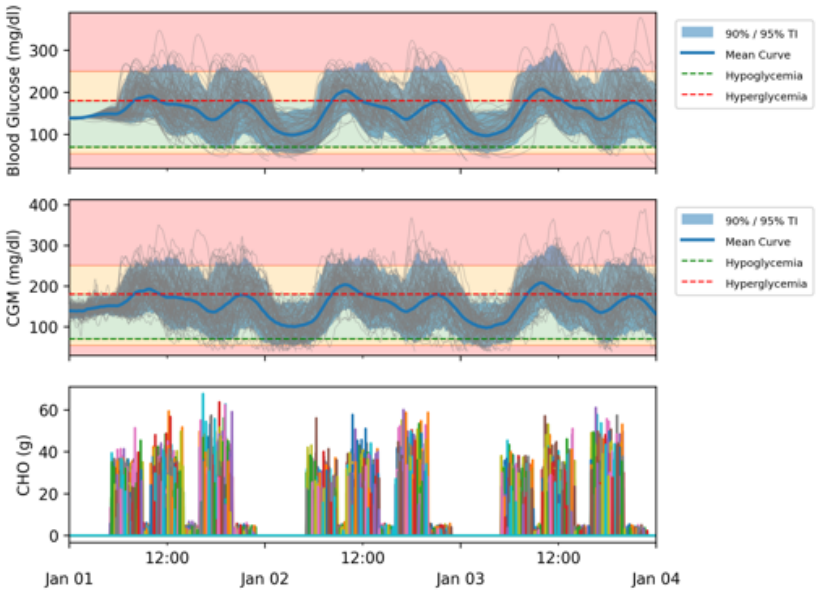


Figure 14: Three-day evaluation of the selected adult-001 symbolic student distilled from the FCL-PPO teacher. The policy was obtained using the `square_threshold_15` PySR configuration and selected based on high reward and low CFR. The figure shows 300 simulated episodes over a three-day evaluation horizon.

Overall, this experiment shows that symbolic regression can still derive compact and meaningful policies in a longer and noisier setting. At the same time, the results demonstrate that symbolic policies are shaped by the reward and evaluation design, and should therefore be assessed carefully under multiple horizons and disturbance conditions before being interpreted as generally robust control strategies.

5.3.5 Performance Summary

In Table 11 an overview can be had of the overall performances of PPO teacher models, distilled SPs and benchmark controllers, given by the mean value with inter-quantile-ranges.

Model	TIR		TBR I		TAR I		CFR		Reward	
HCL-PPO	85.1	[79.7, 92.0]	4.3	[2.7, 7.4]	9.3	[4.0, 13.5]	17.5	[13.2, 31.9]	401.2	[383.5, 419.5]
HCL-SP	83.4	[76.1, 91.7]	3.2	[1.5, 5.1]	11.0	[6.1, 13.9]	8.0	[5.4, 25.9]	401.8	[375.6, 436.2]
BBH	74.5	[73.6, 75.3]	2.3	[1.7, 2.5]	23.5	[21.2, 24.4]	15.2	[12.5, 29.8]	368.7	[358.7, 381.9]
FCL-PPO	88.8	[85.8, 91.9]	3.2	[2.2, 3.8]	8.2	[5.9, 11.0]	11.2	[7.7, 14.3]	419.7	[408.5, 433.0]
FCL-SP	91.3	[78.7, 95.2]	2.6	[1.9, 5.8]	4.2	[0.7, 13.5]	8.3	[2.6, 15.6]	422.2	[381.2, 435.8]
PID	87.1	[79.7, 93.2]	11.9	[5.8, 17.5]	1.0	[0.8, 2.1]	30.2	[22.8, 39.4]	371.4	[326.3, 411.3]

Table 11: Aggregated performance across all patients for each of the proposed control models, compared against the BBH and PID controller baselines. All performance metrics are measured as the average value across the 10 patients, reported with Inter Quantile ranges. Rewards values are based on the Clark Risk reward function. HCL-PPO, HCL-SP, and BBH are open-loop or hybrid closed-loop control methods requiring meals, whilst FCL-PPO, FCL-SP, and PID all are fully closed-loop control methods.

5.4 Case Discussion

5.4.1 Teacher Performance

The PPO teachers achieved performance levels that made them suitable targets for symbolic distillation. Across patients, both HCL and FCL PPO models reached clinically relevant glucose-control performance, with generally high TIR and limited TBR. However, performance varied substantially between virtual patients. Some patients, such as adult-001, adult-008, and adult-009, were controlled effectively, whereas others, particularly adult-004, adult-006, and adult-007, were more difficult to stabilize. This variation suggests that patient-specific glucose-insulin dynamics strongly affected both PPO training and subsequent distillation.

The quality of the teacher was important for the quality of the symbolic student. In both the HCL and FCL settings, symbolic policies generally followed the performance level of their corresponding PPO teachers. Strong teacher policies typically produced strong symbolic policies, while weaker teachers or teachers with high CFR tended to produce less stable symbolic students. This indicates that the symbolic policy was partly bounded by the behavior and state distribution provided by the teacher. If the teacher frequently terminates early or learns unstable dosing behavior, the trajectories and labels available for distillation may be biased or incomplete. Teacher quality should therefore be treated as part of the distillation pipeline, not merely as a preliminary training step.

In the HCL setting, several symbolic policies achieved performance comparable to, or better than, their PPO teachers in terms of CFR. This suggests that symbolic distillation may sometimes smooth or simplify teacher behavior in a beneficial way. However, this was not consistent across all patients. For more difficult patients, the symbolic policies tended to reproduce the limitations of the teacher, indicating that distillation cannot compensate reliably for a weak or unstable teacher policy.

5.4.2 Symbolic Policy Insights

The distilled symbolic policies were generally compact, which is valuable in medical control where interpretability and transparency are important. Unlike the PPO teachers, they can be inspected directly as mathematical dosing rules, showing which variables affect insulin delivery and how. While this does not establish clinical safety, it makes the controllers easier to analyze, stress-test, and constrain.

The simplicity of the final expressions should, however, be interpreted in relation to the design of the state representation. In both the HCL and FCL settings, the symbolic policies primarily relied on CGM and estimated IOB. These variables are not raw unprocessed simulator outputs in the same sense as the full physiological state of the virtual patient. Both CGM and IOB were normalized before being passed to the learning algorithm and IOB was introduced as an engineered summary of recent insulin delivery and residual insulin activity. Thus, IOB provides the policy with a compact representation of delayed insulin effects and a limited form of memory over previous dosing decisions. The resulting expressions are therefore simple partly because relevant temporal information has already been compressed into clinically meaningful state variables.

Although CGM and IOB explained much of the behavior of the distilled policies, this does not imply that the remaining state variables were irrelevant. Meal information in the HCL setting and action history in the FCL setting improved the PPO teachers. Their absence from many symbolic expressions is therefore best interpreted as a result of compression: the student retained the dominant feedback structure needed under the evaluated scenarios, while discarding variables that added complexity without clear performance benefit. This conclusion is conditional on the simulation setup. In the present experiments, meal patterns were structured, and

IOB already summarized recent insulin delivery. Under stronger perturbations, such as skipped meals, larger meal variability, sensor noise, or longer horizons, meal variables and action history may become more important. Their limited use should therefore not be taken as evidence that they are generally unnecessary for glucose control.

Overall, the symbolic policies provide compact and clinically interpretable approximations of the PPO teachers. Their reliance on CGM and IOB is physiologically plausible, but their simplicity should be understood in light of the engineered state variables, restricted operators, action constraints, and high-frequency control. Thus, the main insight is not that glucose control is inherently simple, but that symbolic distillation can reveal plausible dosing rules that remain accessible for further verification under the chosen simulator interface.

5.4.3 Robustness and Scenario Dependence

The robustness experiment showed that strong one-day performance did not necessarily transfer to longer and more variable scenarios. When the evaluation horizon was extended to three days and meal variability was increased, CFR rose substantially, especially for symbolic policies. This suggests that some symbolic students were more sensitive to distribution shifts than their PPO teachers. The three-day traces further indicated that many failures occurred during overnight periods, where no meals were available to counteract excessive insulin delivery.

This result is important because the training setup used structured daily meal scenarios. Even when meal timing and carbohydrate amount were perturbed, meals still occurred within relatively regular daily patterns. A policy may therefore perform well by implicitly relying on the expectation that meals will occur soon, rather than by learning a generally safe dosing strategy. Such behavior can improve reward in the training environment but may become unsafe under delayed meals, missed meals, or longer fasting periods.

The symbolic policies make this issue easier to identify. For example, action-history-based FCL policies or HCL policies that use time-related variables may indicate dependence on the structure of the scenario rather than purely feedback-based control. This does not mean that the simulator is invalid, but it shows that high in-silico performance should be interpreted in relation to the specific meal scenario, action constraints, reward function, and termination criteria used during evaluation.

Another source of uncertainty is the choice of PySR configuration. In this thesis, the symbolic-regression configuration was selected from a pilot experiment and then fixed for the remaining patients to keep the evaluation computationally feasible and comparable. However, the pilot results showed that several configurations achieved relatively similar performance, suggesting that the selected configuration should not be interpreted as uniquely optimal.

This limitation was also indicated by an additional multi-day evaluation on a single patient, where threshold-based symbolic policies performed better than the simpler linear configuration. This suggests that configurations selected under one-day evaluation conditions may not remain optimal under longer-horizon or more variable scenarios. With more computational resources, it would therefore be relevant to evaluate several PySR configurations per patient.

5.4.4 Clinical Interpretability and Limitations

The diabetes case study shows that SPID can extract compact symbolic insulin-dosing rules from PPO teachers, and that these rules can sometimes retain strong glucose-control performance. This is promising because symbolic policies are easier to inspect, compare, and constrain than neural network policies. In particular, threshold-based policies depending on CGM and IOB provide dosing rules that can be directly visualized and evaluated over clinically relevant state ranges.

However, interpretability does not imply safety. A simple symbolic expression may still encode brittle or unsafe behavior, especially if it exploits simulator regularities or omits important state variables. Low per-step insulin outputs should also not be interpreted as sufficient evidence of safety, since actions were taken every three minutes and repeated small doses can accumulate over time. The symbolic policies should therefore be regarded as interpretable candidate controllers and diagnostic tools, not as clinically validated dosing algorithms.

Overall, the diabetes case study demonstrates both the promise, pitfalls and the limitation of symbolic policy distillation for medical control. SPID can reveal compact and physiologically plausible feedback structures, particularly through the consistent use of CGM and IOB. At the same time, the method remains dependent on teacher quality, state design, reward design, and simulator conditions. The strongest conclusion is therefore not that glucose control can be solved by simple symbolic rules, but that symbolic distillation can expose useful, interpretable approximations of DRL controllers and highlight where these controllers may fail under more realistic stress conditions.

6. Discussion

This thesis extended SPID to two applied control problems in order to evaluate its behavior outside standard benchmark environments and investigate its practical limitations. In both the drone and glucose-control case studies, PPO teachers were distilled into substantially simpler symbolic policies that often preserved comparable task performance. These results motivate a broader discussion of when symbolic distillation is useful, what it reveals about learned controllers, and where its limitations arise.

6.1 Performance Is Conditional on the Teacher

The performance of the symbolic student is conditional on the quality and coverage of the teacher policy. SPID does not learn an optimal policy independently; it distills behavior from states visited by the trained teacher and by the student-teacher mixture during data aggregation. If the teacher has not learned robust behavior in parts of the state space, or if these regions are rarely visited during training, the symbolic student has limited opportunity to recover reliable actions there.

This dependency appeared in both case studies. In the diabetes case, teachers with high CFR generated incomplete trajectories that provided limited information about delayed insulin effects and recovery after meals. Distilling from such teachers can produce compact symbolic policies, but these policies may inherit an unstable control strategy or a biased state distribution. In the drone case, the teacher achieved high return but produced oscillatory motor commands, likely because the reward did not penalize action fluctuations near the target. The symbolic student recovered a smoother PD-like rule with comparable performance, suggesting that distillation can sometimes regularize teacher-specific action noise rather than reproduce it exactly.

Teacher variability also contributes to student variability. If PPO converges differently across patients or runs, the trajectories used for distillation may cover different regions of the state space, include different failure modes, or reflect different control strategies. The symbolic regression step then compresses this sampled behavior, meaning that small differences in teacher trajectories can lead to different symbolic expressions and evaluation outcomes. Teacher convergence, robustness, and critical-failure rate should therefore be treated as part of the distillation pipeline, not merely as preliminary reinforcement learning training results.

6.2 Interpretability and Insight

Across both case studies, symbolic distillation provided more than model compression. The resulting expressions made it possible to inspect which state variables were used by the learned controllers and how they influenced the action. In the drone case, the distilled policy depended mainly on altitude, vertical velocity, and previous motor output, resembling a classical feedback controller. In the diabetes case, many symbolic policies relied primarily on CGM and estimated IOB, which are clinically meaningful variables for insulin dosing.

However, interpretability should not be equated with safety. A compact symbolic expression can be easier to inspect, but it can still encode undesirable or brittle behavior. This was visible in the glucose-control case, where some symbolic policies achieved high reward despite relying on simulator regularities such as predictable meal timing or previous action patterns. A different safety concern appeared in the drone case, where the symbolic student did not automatically inherit the teacher's action bounds and could produce motor outputs outside the intended range.

The interpretability benefit of SPID should therefore be understood as diagnostic rather than conclusive. Symbolic policies can make learned strategies easier to inspect, but they still require stress testing, domain evaluation, and explicit safety constraints before they can be considered reliable.

6.3 The Role of Environment and Reward Design

Reward and environment design shape both the neural teacher and the symbolic student. The reward function determines which behavior the PPO teacher is encouraged to learn, and the student can only distill behavior made available by that teacher. If the reward is sparse, too flat, or poorly aligned with the intended objective, the teacher may learn strategies that achieve high return without representing the desired control behavior. These limitations can then be inherited, approximated, or made more visible by the symbolic student.

This was observed in both case studies. In the drone task, the reward was nearly flat near the hover target, allowing oscillatory motor commands to achieve high return as long as the drone remained close to the target altitude. In the diabetes task, regular meal patterns may allow policies to dose insulin in anticipation of expected meals. Such behavior can perform well under the simulated meal scenario, but may become unsafe if meals are delayed, skipped, or announced incorrectly. Strong simulator performance therefore does not necessarily imply robust control behavior.

The environment interface also affects what kind of symbolic policy can be recovered. PPO benefits from normalized observations, bounded actions, and action mappings that stabilize optimization. However, these choices may not be optimal for symbolic regression. For example, a highly nonlinear action mapping can make the symbolic policy harder to interpret as a direct physical control rule, while overly restrictive action representations may force the student to approximate the interface rather than the underlying control problem.

Environment design should therefore be treated as part of the distillation pipeline. The reward function, state representation, action mapping, simulator scenarios, and symbolic search space jointly determine whether the student policy is accurate, interpretable, numerically stable, and meaningful over the relevant state domain.

6.4 Sufficiency of Simple Policies

A recurring result across both case studies was that relatively simple symbolic expressions could match, and in some cases exceed, the performance of the neural teachers. This raises an important question: whether SPID is extracting compact structure learned by the DRL policy, or

whether the environment and state representation already admit a simpler solution.

These interpretations are not mutually exclusive. DRL may act as a useful search procedure for discovering effective behavior when the underlying rule is simple but not obvious *a priori*. In this case, symbolic distillation extracts the compact control rule implicitly found by the neural policy. Conversely, if low-complexity symbolic policies consistently achieve similar performance, this may indicate that complex neural policies are not always necessary for the formulated task.

This was visible in both case studies. The drone task reduced to a compact feedback rule resembling classical control, while the diabetes policies often relied on a reduced set of variables, mainly CGM and estimated IOB. These findings suggest that SPID can function as a diagnostic tool for assessing whether the complexity of a DRL controller is justified. However, this conclusion is limited to the structured simulated environments considered here, where the dynamics are low-dimensional and partly governed by known physical or physiological relationships.

6.5 Practical Usability and Limits of SPID

A central practical observation is that SPID works best when the reinforcement learning problem can be represented as a structured, low-dimensional control task with meaningful state variables. In such settings, symbolic regression can produce compact policies with relatively limited task-specific redesign. These policies can either be considered candidate controllers or used as post-hoc approximations for inspecting what the neural policy has learned.

The dependence on symbolic regression also defines the method's limits. As the number of variables, operators, and allowed expression complexity increases, the search space grows rapidly and interpretability decreases. Richer operator sets may improve fidelity, but can also introduce unstable or difficult-to-constrain expressions, which is especially problematic in safety-critical domains.

SPID is therefore best suited to simulated control problems where repeated rollouts are feasible, the action space is compatible with symbolic modelling, and the state variables are physically or clinically meaningful. It is less directly applicable to high-dimensional observations, poorly structured state spaces, or domains where simulator interaction is costly or unsafe. Overall, SPID should be viewed as a targeted method for interpretable control and policy diagnosis, not as a general replacement for DRL.

7. Conclusion

This thesis investigated whether Symbolic Policy Interpretable Distillation (SPID) can extract compact and interpretable control policies from Deep Reinforcement Learning (DRL) teachers. We implemented the SPID framework, reproduced selected benchmark experiments, and applied the method to two control tasks: quadcopter hover control and in-silico blood glucose control for Type 1 diabetes.

The benchmark reproduction showed that the implementation could recover symbolic policies with performance close to the neural teachers in several standard environments. In the drone case study, SPID recovered a compact feedback-like controller depending mainly on altitude, vertical velocity, and previous motor output. This showed that symbolic distillation can recover physically meaningful control structure from a neural policy, while also revealing practical issues

such as reward sensitivity and the need to explicitly enforce action bounds.

In the diabetes case study, several symbolic insulin-dosing policies achieved performance comparable to their PPO teachers and often relied on clinically meaningful variables such as CGM and estimated IOB. However, the results also showed that symbolic students remained dependent on teacher quality, reward design, state representation, action mapping, and simulator conditions. Strong one-day performance did not always transfer to longer and more variable scenarios, indicating that some policies may have exploited regularities in the simulated environment.

Overall, SPID provided both policy compression and insight into learned DRL behavior. The extracted expressions made it possible to inspect which variables influenced the controllers and to identify simple feedback structures or brittle strategies. However, symbolic interpretability is not equivalent to safety. The main conclusion is therefore that SPID is most useful as a tool for interpretable control and policy diagnosis in structured, low-dimensional simulation settings, while safety-critical deployment would require further robustness testing, explicit constraints, and domain-specific validation.

8. Perspectives and Future Work

8.1 A Deeper Dive into Policy Distillation

This thesis has explored Symbolic Policy Distillation and its potential as a method for deriving interpretable policies from deep reinforcement learning controllers. However, several aspects of the method remain open for further investigation. Future work could examine extensions of the distillation framework, provide a deeper understanding of how symbolic regression parameters influence the resulting policies, and evaluate robustness under broader sources of uncertainty and distributional shift. In particular, the trade-off between policy performance, fidelity to the teacher model, and interpretability remains an important direction for future study.

8.2 Safety-Aware Symbolic Distillation

Since the distillation objective is explicitly defined through a loss function, additional terms could be introduced to guide the symbolic student toward desired behavior. In the diabetes-control setting, such terms could penalize abrupt dose changes, excessive insulin delivery, or actions that increase the risk of hypoglycaemia. This would make symbolic distillation a more flexible framework, where the student is encouraged to retain useful behavior from the teacher while satisfying additional safety- or domain-specific preferences.

Another related direction is the use of explicit safety shields. In this thesis, the implementation allowed for simple shielding mechanisms, and preliminary tests were conducted. However, these initial experiments did not show clear improvements over policies trained with a carefully designed reward function and conservative action constraints. In particular, improving the reward formulation and insulin action mapping had a stronger effect on both teacher and student performance than adding a simple post-hoc shield. For this reason, safety shields were not explored further in the final experiments.

This should not be interpreted as evidence that shielding is unimportant. Rather, it suggests that simple shields may be insufficient if the underlying reward function, state representation, or teacher policy already encourages undesirable behavior. A shield can prevent some unsafe actions, but it does not necessarily teach the policy a better long-term control strategy. In safety-critical settings, shields may therefore be most useful when combined with reward design, constrained action spaces, and safety-aware distillation losses. Since symbolic policies are analytical functions, future work could investigate whether maximum insulin delivery can be bounded over valid state ranges, whether insulin-on-board can be used to impose state-dependent dose caps, or whether runtime monitors can reject unsafe symbolic policy outputs before they are passed to the simulator. Such shields may not necessarily improve average reward, but they could improve worst-case safety and robustness under conditions not represented during training.

A related direction is safety-aware data selection during distillation. In preliminary experiments with weaker teacher models, symbolic students could sometimes be substantially improved by filtering teacher trajectories and using only rollouts that completed the full evaluation horizon. This suggests that successful teacher trajectories may provide a cleaner training signal than trajectories ending in critical failure. However, filtering should be used carefully, since removing all unsafe trajectories may also remove information about states where recovery behavior is needed. Future work could therefore investigate trajectory weighting or safety-filtered dataset aggregation, where successful trajectories are prioritized while unsafe states are still included with appropriate penalties.

8.3 Robustness Under Realistic Simulation Conditions

Another important extension is robustness across longer and more variable simulations. The present experiments were based on relatively short simulated control periods and controlled meal scenarios. Training and distilling policies over multiple days, with greater variation in meal timing, carbohydrate estimation, sensor noise, and patient behavior, may reduce the risk of policies exploiting narrow simulator regularities. This could make both the neural teachers and symbolic students less dependent on a specific daily pattern and improve robustness under more realistic conditions. A simple but powerful improvement would also be to randomize when the episode begins, this would make it harder to overfit.

Future work should also evaluate symbolic policies and PPO teachers under stronger out-of-distribution conditions. This could include unseen meal schedules, larger meal-size errors, missed meals, sensor noise, and patient-specific disturbances not present during training. Such experiments would help determine whether the models have learned robust glucose-control behavior or whether they primarily reproduce regularities of the training scenario.

8.4 Patient-Adaptive Symbolic Policies

Future work could investigate multi-patient symbolic distillation. In this thesis, patient-specific controllers were trained and distilled separately. An alternative would be to train a single DRL policy across multiple virtual patients and include patient characteristics, such as body weight, in the state representation. Since basal-bolus parameters such as basal rate, correction factor, and carbohydrate ratio are partly informed by patient-specific physiological characteristics, a symbolic student trained across patients might recover policy structures resembling adaptive basal-bolus rules. Such a model would not necessarily identify clinical parameters directly, since these ratios are not determined by body weight alone. However, it could provide an interpretable population-level dosing rule that adjusts according to patient features and may reveal how the DRL policy uses inter-patient variability.

8.5 Symbolic Distillation as an Analysis Tool

In this thesis, PPO checkpoints were saved at regular training intervals, although only the best models were used for the main distillation results. Distilling intermediate checkpoints could make the learning process more transparent by showing how control strategies evolve over training, for example from simple basal-like behavior toward meal responses, correction strategies, or patient-specific feedback rules. It might also help identify when the teacher begins to exploit undesirable simulator regularities.

9. Data and Material Availability

Git repository of the project

Code repository for the project GitHub

Git repository of the Simglucose enviroment

Code repository for the Simglucose enviroment GitHub

SPID

The original implementation associated with the SPID paper is not publicly available at the time of writing. The SPID implementation used in this thesis was therefore developed independently based on the paper and correspondence with the author.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework, 2019.

[2] Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983.

[3] Osbert Bastani, Yewen Pu, and Armando Solar-Lezama. Verifiable reinforcement learning via policy extraction. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.

[4] Sumana Basu, Marc-André Legault, Adriana Romero-Soriano, and Doina Precup. On the challenges of using reinforcement learning in precision drug dosing: Delay and prolongedness of action effects, 2023.

[5] Tadej Battelino, Thomas Danne, Richard M. Bergenstal, Stephanie A. Amiel, Roy Beck, et al. Clinical targets for continuous glucose monitoring data interpretation: Recommendations from the international consensus on time in range. *Diabetes Care*, 42(8):1593–1603, 06 2019.

[6] Cari Berget, Laurel H. Messer, and Gregory P. Forlenza. A clinical overview of insulin pump therapy for the management of diabetes: Past, present, and future of intensive therapy. *Diabetes Spectrum*, 32(3):194–204, 2019.

[7] Man CD, Micheletto F, Lv D, Breton M, Kovatchev B, and Cobelli C. The uva/padova type 1 diabetes simulator: New features. *J Diabetes Sci Technol*, 2014.

[8] Jane L. Chiang, M. Sue Kirkman, Lori M.B. Laffel, Anne L. Peters, and on behalf of the Type 1 Diabetes Sourcebook Authors. Type 1 diabetes through the life span: A position statement of the american diabetes association. *Diabetes Care*, 37(7):2034–2054, 06 2014.

[9] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning, 2016.

[10] Miles Cranmer. Interpretable machine learning for science with pysr and symbolicregression.jl, 2023.

[11] Paul C. Davidson, Heather R. Hebblewhite, Robert D. Steed, and Bruce W. Bode. Analysis of guidelines for basal-bolus insulin dosing: Basal insulin, correction factor, and carbohydrate-to-insulin ratio. *Endocrine Practice*, 14(9):1095–1101, December 2008.

[12] Ketan K. Dhatariya, Nicole S. Glaser, Ethel Codner, and Guillermo E. Umpierrez. Diabetic ketoacidosis. *Nature Reviews Disease Primers*, 6(1):40, 2020.

[13] Lehel Dénes-Fazakas, László Szilágyi, Levente Kovács, Andrea De Gaetano, and György Eigner. Reinforcement learning: A paradigm shift in personalized blood glucose management for diabetes. *Biomedicines*, 12(9), 2024.

[14] Daniela Elleri, David B. Dunger, and Roman Hovorka. Closed-loop insulin delivery for treatment of type 1 diabetes. *BMC Medicine*, 9:120, 2011.

[15] Ian Fox, Joyce Lee, Rodica Pop-Busui, and Jenna Wiens. Deep Reinforcement Learning for Closed-Loop Blood Glucose Control. *arXiv e-prints*, page arXiv:2009.09051, September 2020.

[16] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. *arXiv preprint arXiv:1802.09477*, 2018.

[17] Weiwei Gu and Senquan Wang. An improved strategy for blood glucose control using multi-step deep reinforcement learning, 2024.

[18] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018.

[19] J. Arthur Harris and Francis G. Benedict. A biometric study of human basal metabolism. *Proceedings of the National Academy of Sciences*, 4(12):370–373, 1918.

[20] Daniel Hein, Steffen Udluft, and Thomas A. Runkler. Interpretable policies for reinforcement learning by genetic programming, 2018.

[21] Chirath Hettiarachchi, Nicolo Malagutti, Christopher Nolan, Eleni Daskalaki, and Hanna Suominen. A reinforcement learning based system for blood glucose control without carbohydrate estimation in type 1 diabetes: In silico validation. *Proceedings of the 2022 44th Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC)*, pages 950–956, 2022.

[22] Peilang Li, Umer Siddique, and Yongcan Cao. Symbolic policy distillation for interpretable reinforcement learning. *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025.

[23] Yuxi Li. Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274*, 2018.

[24] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2019.

[25] Stephanie Milani, Zhicheng Zhang, Nicholay Topin, Zheyuan Ryan Shi, Charles Kamhoua, Evangelos E. Papalexakis, and Fei Fang. Maviper: Learning decision tree policies for interpretable multi-agent reinforcement learning, 2022.

[26] Seung Joon Moon, Insup Jung, and Choong Yeol Park. Current advances of artificial pancreas systems: A comprehensive review of the clinical evidence. *Diabetes & Metabolism Journal*, 45(6):813–839, 2021.

[27] National Institute of Diabetes and Digestive and Kidney Diseases. Low blood glucose (hypoglycemia). https://www.niddk.nih.gov/health-information/diabetes/overview/preventing-problems/low-blood-glucose-hypoglycemia, 2021. Accessed: 2026-05-15.

[28] Jacopo Panerati, Hehui Zheng, SiQi Zhou, James Xu, Amanda Prorok, and Angela P. Schoellig. Learning to fly—a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021.

[29] J. C. Pickup and A. J. Sutton. Severe hypoglycaemia and glycaemic control in type 1 diabetes: meta-analysis of multiple daily insulin injections compared with continuous subcutaneous insulin infusion. *Diabetic Medicine*, 25(7):765–774, 2008.

[30] Michael C. Riddell, Dessi P. Zaharieva, Loren Yavelberg, Ali Cinar, and Veronica K. Jamnik. Exercise and the development of the artificial pancreas: One of the more difficult series of hurdles. *Journal of Diabetes Science and Technology*, 9(6):1217–1226, 2015.

[31] Stephane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Geoffrey Gordon, David Dunson, and Miroslav Dudík, editors, *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635, Fort Lauderdale, FL, USA, 11–13 Apr 2011. PMLR.

[32] Andrei A. Rusu, Sergio Gomez Colmenarejo, Caglar Gulcehre, Guillaume Desjardins, James Kirkpatrick, Razvan Pascanu, Volodymyr Mnih, Koray Kavukcuoglu, and Raia Hadsell. Policy distillation, 2016.

[33] Michael Schmidt and Hod Lipson. Distilling free-form natural laws from experimental data. *Science*, 324(5923), 2009.

[34] John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization. *arXiv preprint arXiv:1502.05477*, 2017.

[35] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2018.

[36] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[37] Andrew Silva, Taylor Killian, Ivan Jimenez, Sung-Hyun Son, and Matthew Gombolay. Optimization methods for interpretable differentiable decision trees applied to reinforcement learning. In Silvia Chiappa and Roberto Calandra, editors, *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, volume 108 of *Proceedings of Machine Learning Research*, pages 1855–1865. PMLR, 26–28 Aug 2020.

[38] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2020.

[39] Andreas Thomas and Lutz Heinemann. Algorithms for automated insulin delivery: An overview. *Journal of Diabetes Science and Technology*, 16(5):1228–1238, 2022. PMID: 33955251.

[40] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2025.

[41] Abhinav Verma, Vijayaraghavan Murali, Rishabh Singh, Pushmeet Kohli, and Swarat Chaudhuri. Programmatically interpretable reinforcement learning, 2019.

[42] Marco Virgolin and Solon P. Pissis. Symbolic regression is np-hard, 2022.

[43] Phanuphong Viroonluecha, Esteban Egea-Lopez, and Jose Santa. Evaluation of blood glucose level control in type 1 diabetic patients using deep reinforcement learning. *PLOS ONE*, 17(9):e0274608, 2022.

[44] George A. Vouros. Explainable deep reinforcement learning: State of the art and challenges. *ACM Computing Surveys*, 55(5):1–39, December 2022.

[45] World Health Organization. Diabetes. `https://www.who.int/news-room/fact-sheets/detail/diabetes`, 2024. Accessed: 2026-05-15.

[46] Jinyu Xie. Simglucose v0.2.1. `https://github.com/jxx123/simglucose`, 2018.

[47] Zhao Yanfeng, Jun Kit Chaw, Chaw Id, Mei Choo Ang, Ang Id, Yiqi Tew Tar Umt, Xiao Shi, Lin Liu, and Xiang Cheng. A safe-enhanced fully closed-loop artificial pancreas controller based on deep reinforcement learning. *PLOS One*, 20, 01 2025.

[48] Xia Yu, Zi Yang, Xiaoyu Sun, Hao Liu, Hongru Li, Jingyi Lu, Jian Zhou, and Ali Cinar. Deep reinforcement learning for automated insulin delivery systems: Algorithms, applications, and prospects. *AI*, 6(5), 2025.

[49] Karl J. Åström and Tore Hägglund. *Advanced PID Control*. Instrumentation, Systems, and Automation Society, Research Triangle Park, NC, 2006.

Appendices

A. Drone Complexity Plots

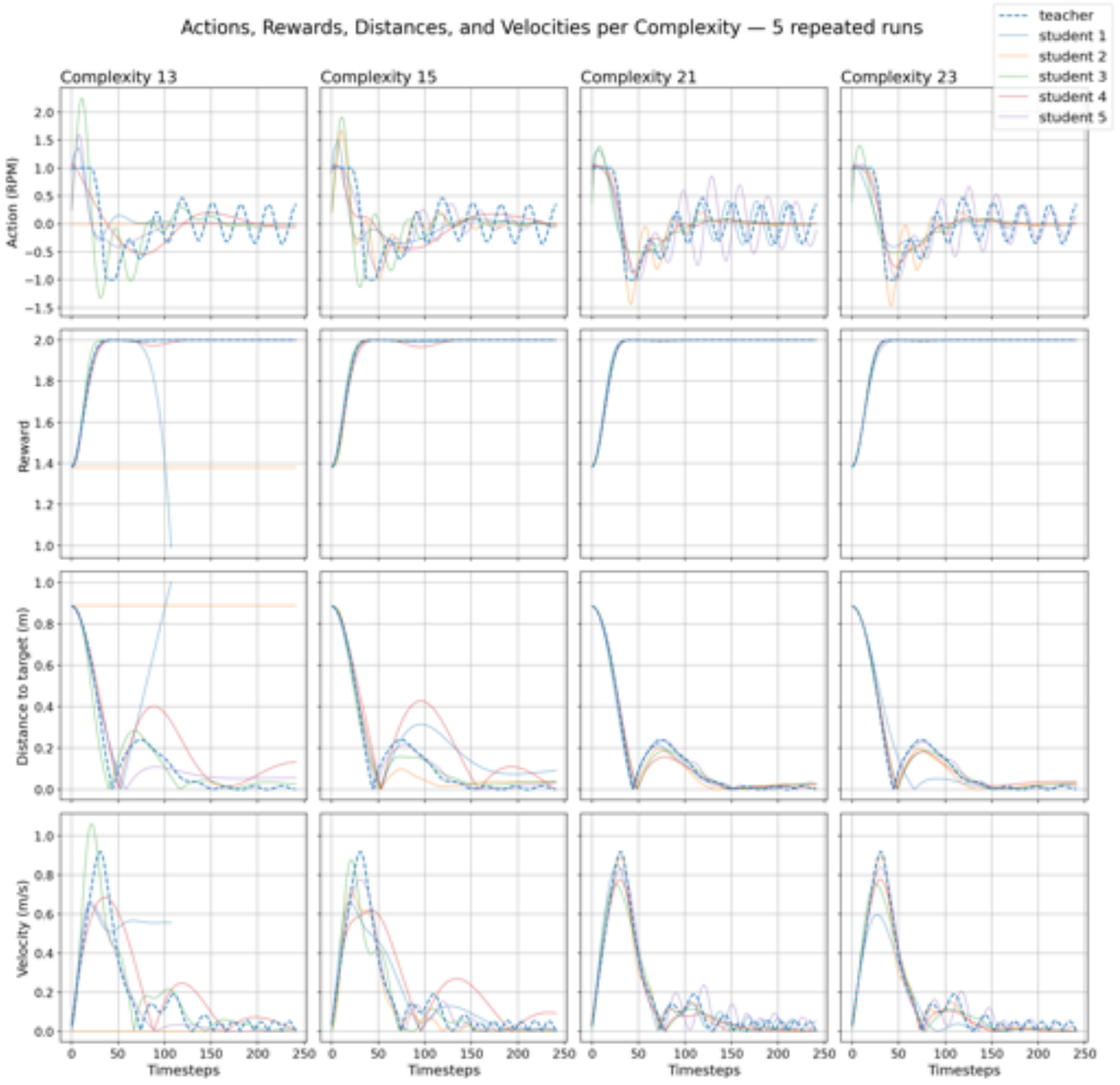


Figure 15: Actions, rewards, distances, and velocities for five repeated distillation runs under identical configurations across four complexity levels (13, 15, 21, 23), using 20,000 timesteps and `maxsize` 30 with simple operators.

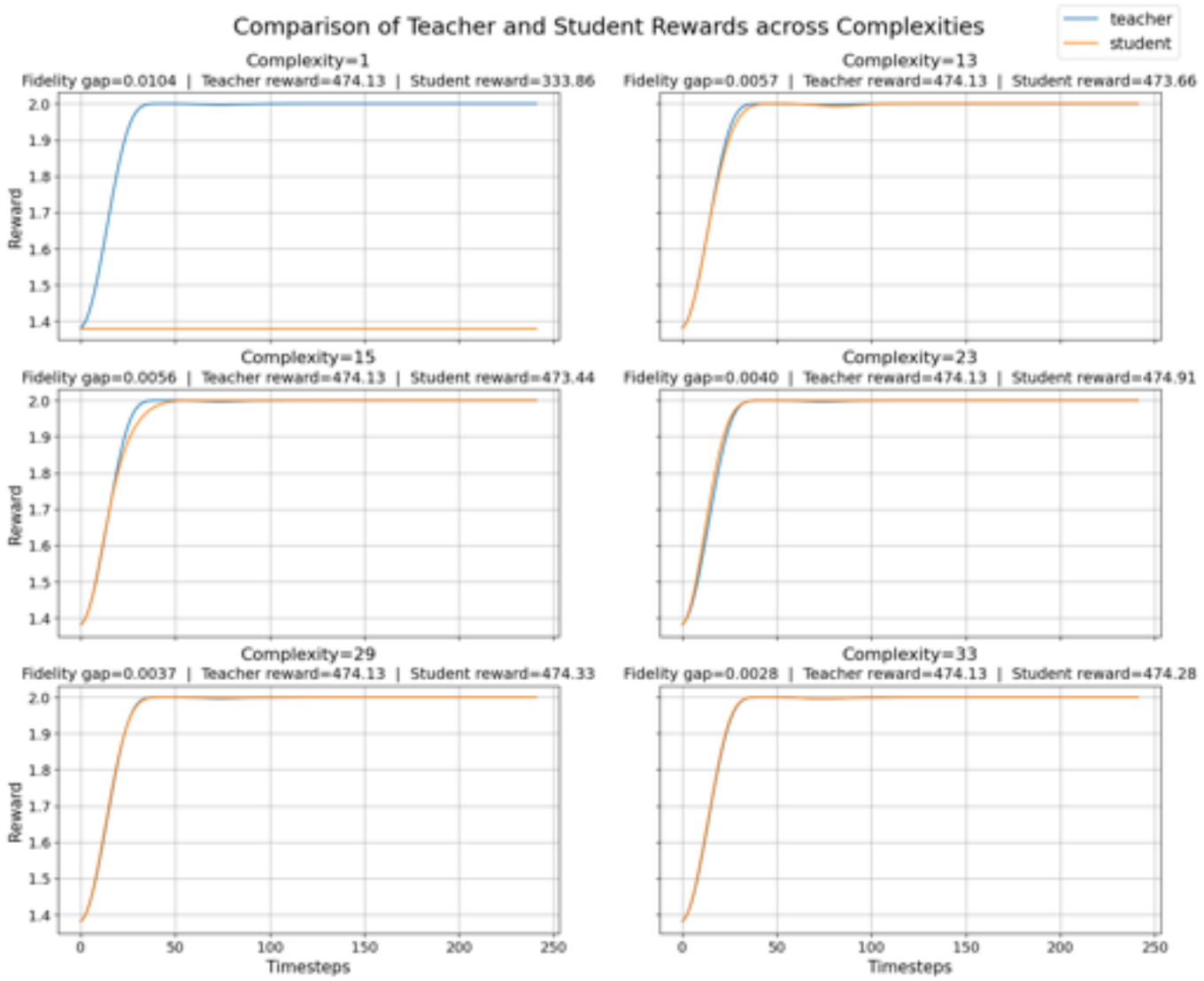


Figure 16: Comparison of teacher and student reward curves across six expression complexity levels, using 20,000 timesteps, `maxsize` 40, and simple operators.

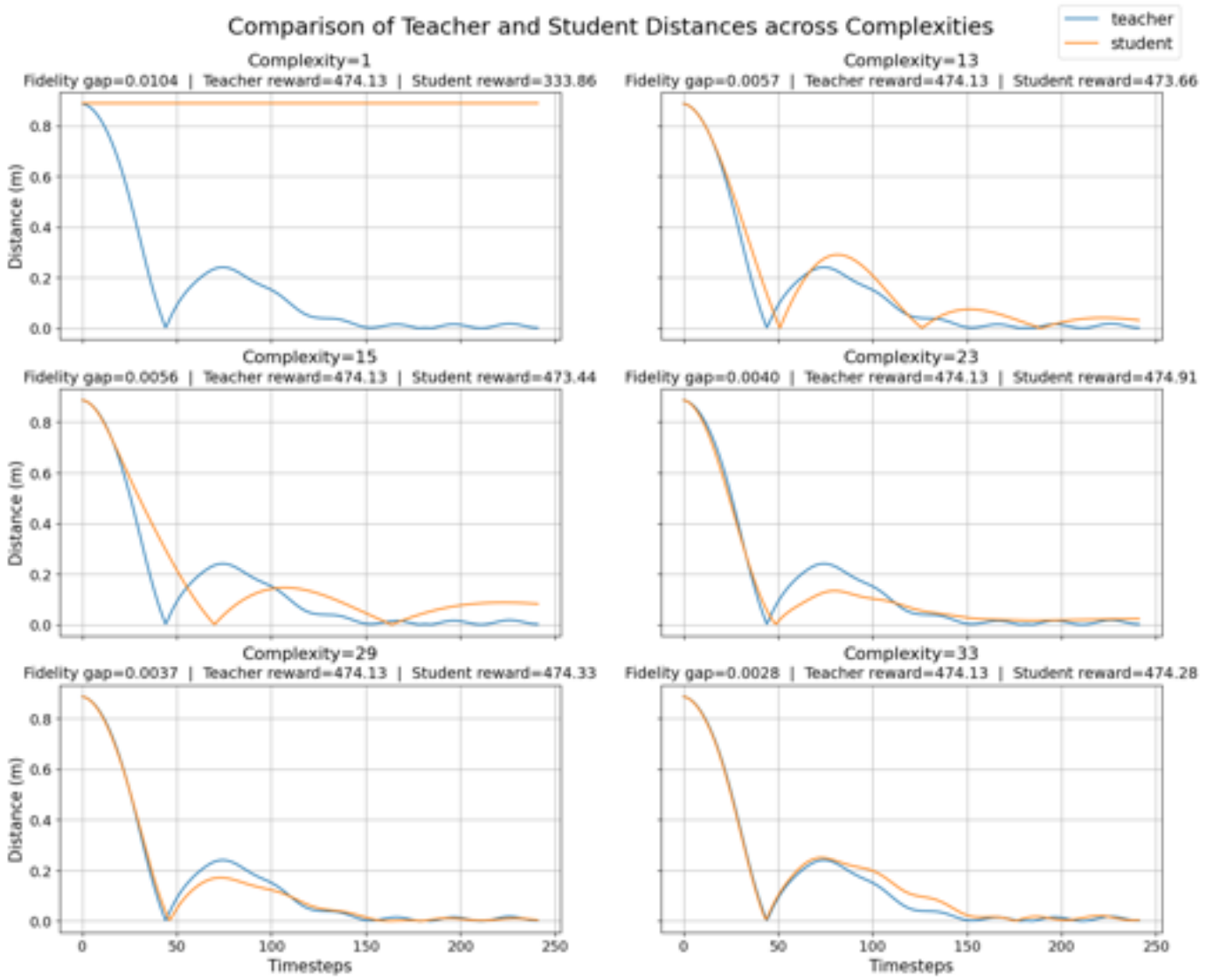


Figure 17: Comparison of teacher and student distance-to-target curves across six expression complexity levels, using 20,000 timesteps, `maxsize` 40, and simple operators.

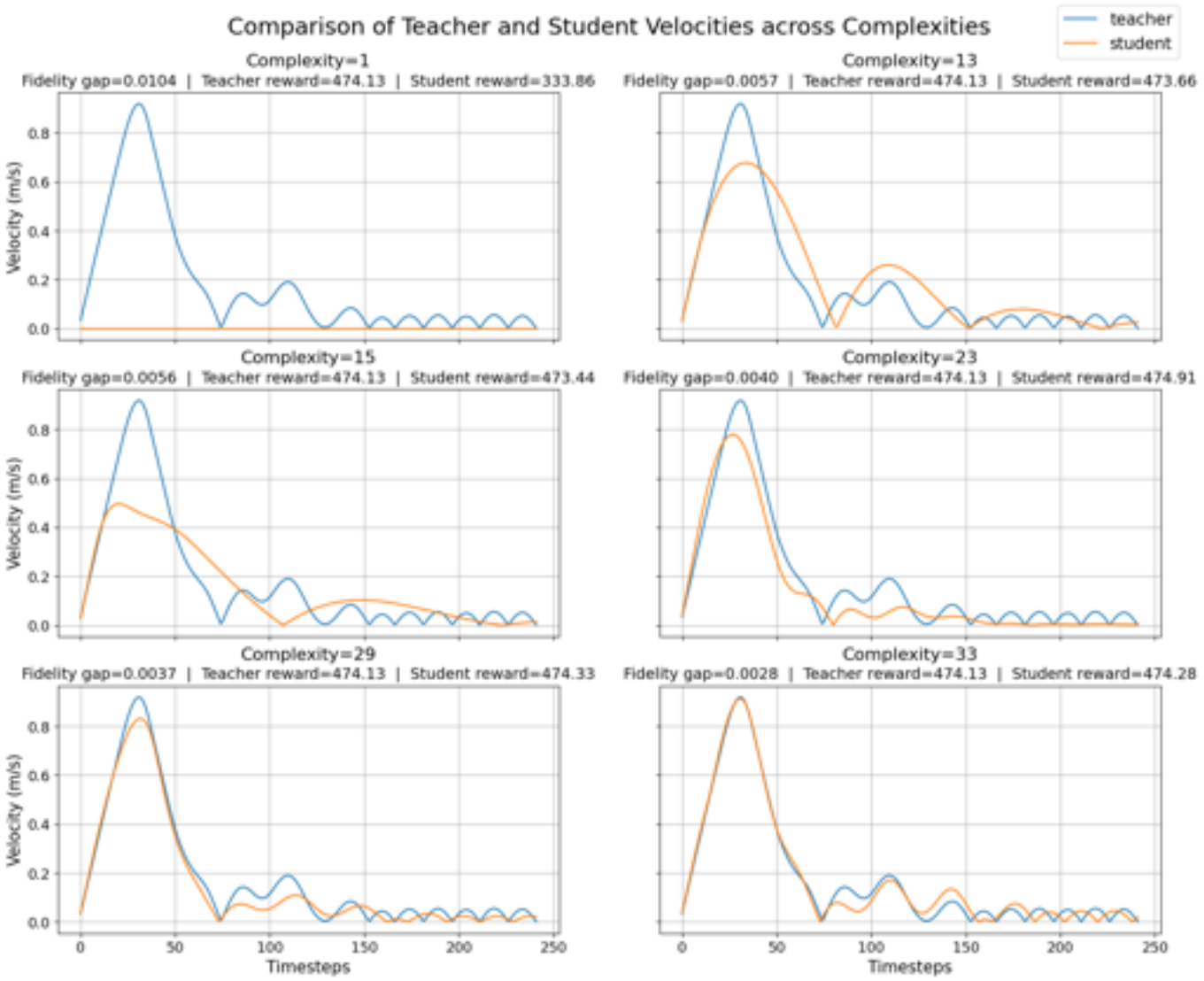


Figure 18: Comparison of teacher and student vertical velocity curves across six expression complexity levels, using 20,000 timesteps, `maxsize` 40, and simple operators.

B. Blood Glucose Tolerance Plots

B.1 Blood Glucose Tolerance Plots PPO Fully Closed-Loop

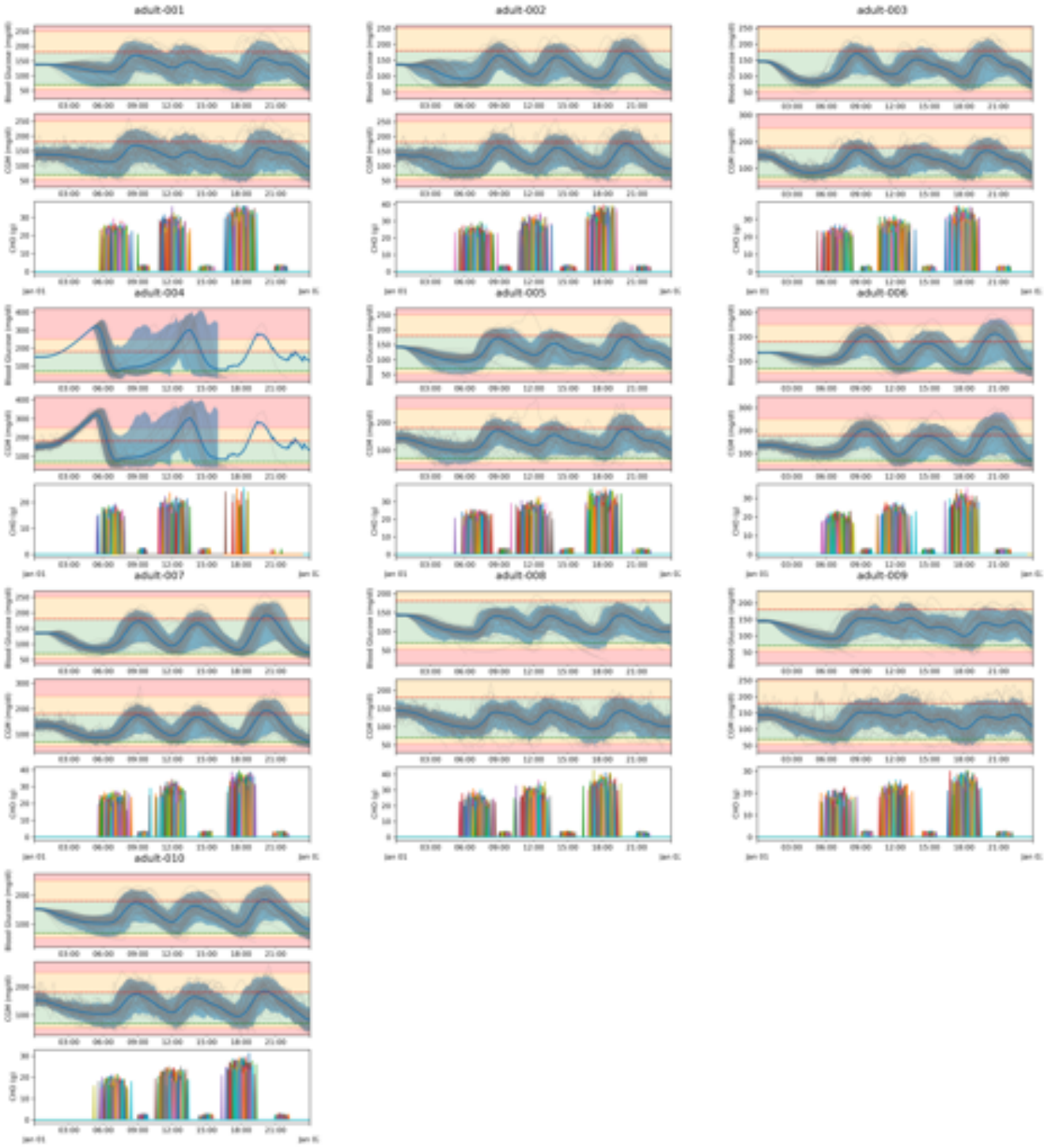


Figure 19: Twenty-four-hour evaluation traces for the FCL action-history PPO controllers across all adult virtual patients. Each patient panel contains BG traces, CGM traces, and carbohydrate disturbances. Thin blue lines show individual evaluation episodes, the darker blue line shows the mean trajectory, and the shaded blue region shows the empirical tolerance interval. The colored background indicates the corresponding glycemic zones. The carbohydrate bars show meal and snack events. The figure highlights both differences between virtual patients and variability caused by random meal timing and carbohydrate amounts.

B.2 Blood Glucose Tolerance Plots PPO Hybrid Closed-Loop

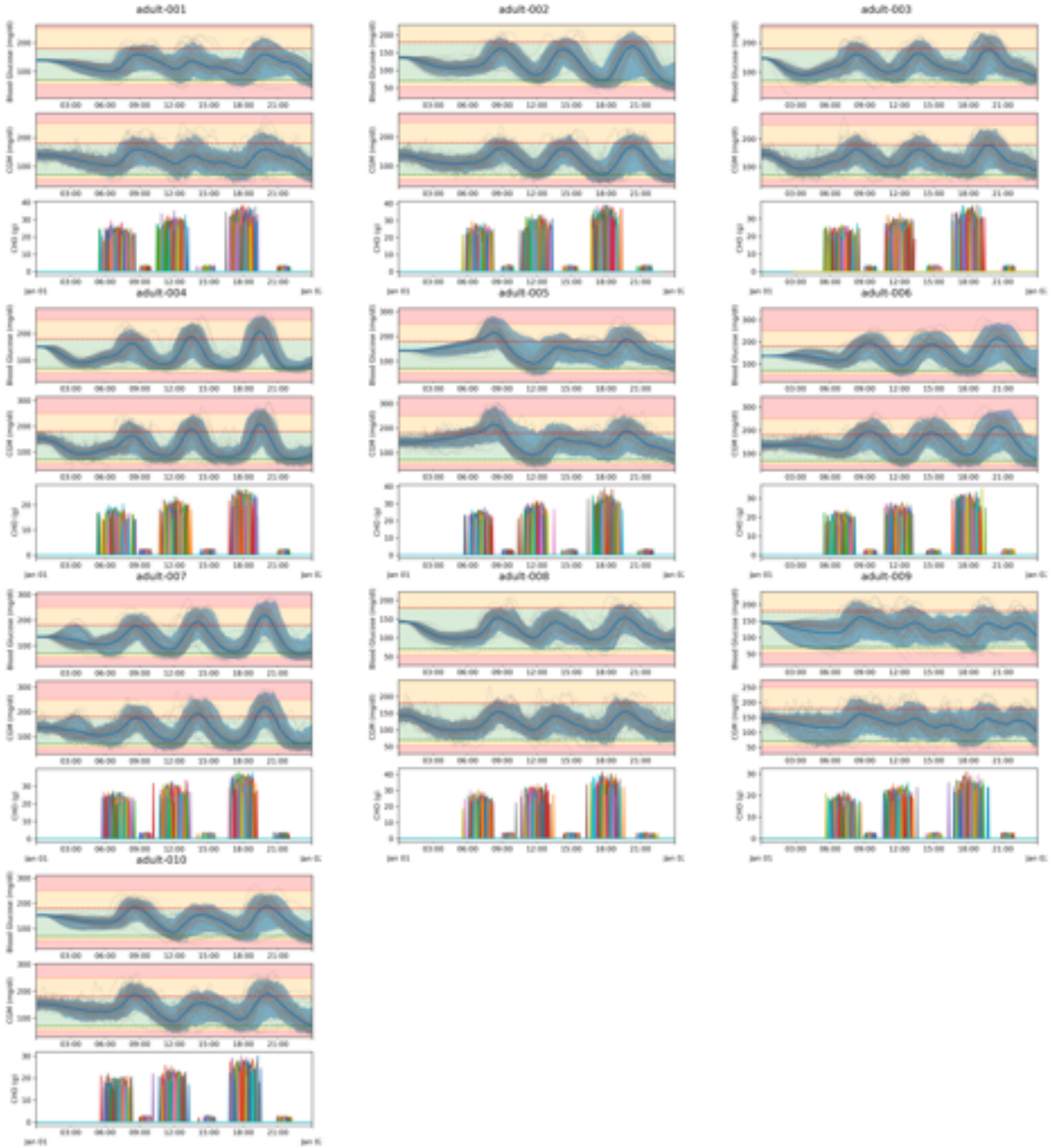


Figure 20: Twenty-four-hour evaluation traces for the HCL action-history PPO controllers across all adult virtual patients. Each patient panel contains BG traces, CGM traces, and carbohydrate disturbances. Thin blue lines show individual evaluation episodes, the darker blue line shows the mean trajectory, and the shaded blue region shows the empirical tolerance interval. The colored background indicates the corresponding glycemic zones. The carbohydrate bars show meal and snack events. The figure highlights both differences between virtual patients and variability caused by random meal timing and carbohydrate amounts.

C. Learning Curves

C.1 Learning curves for Hybrid Closed-Loop model

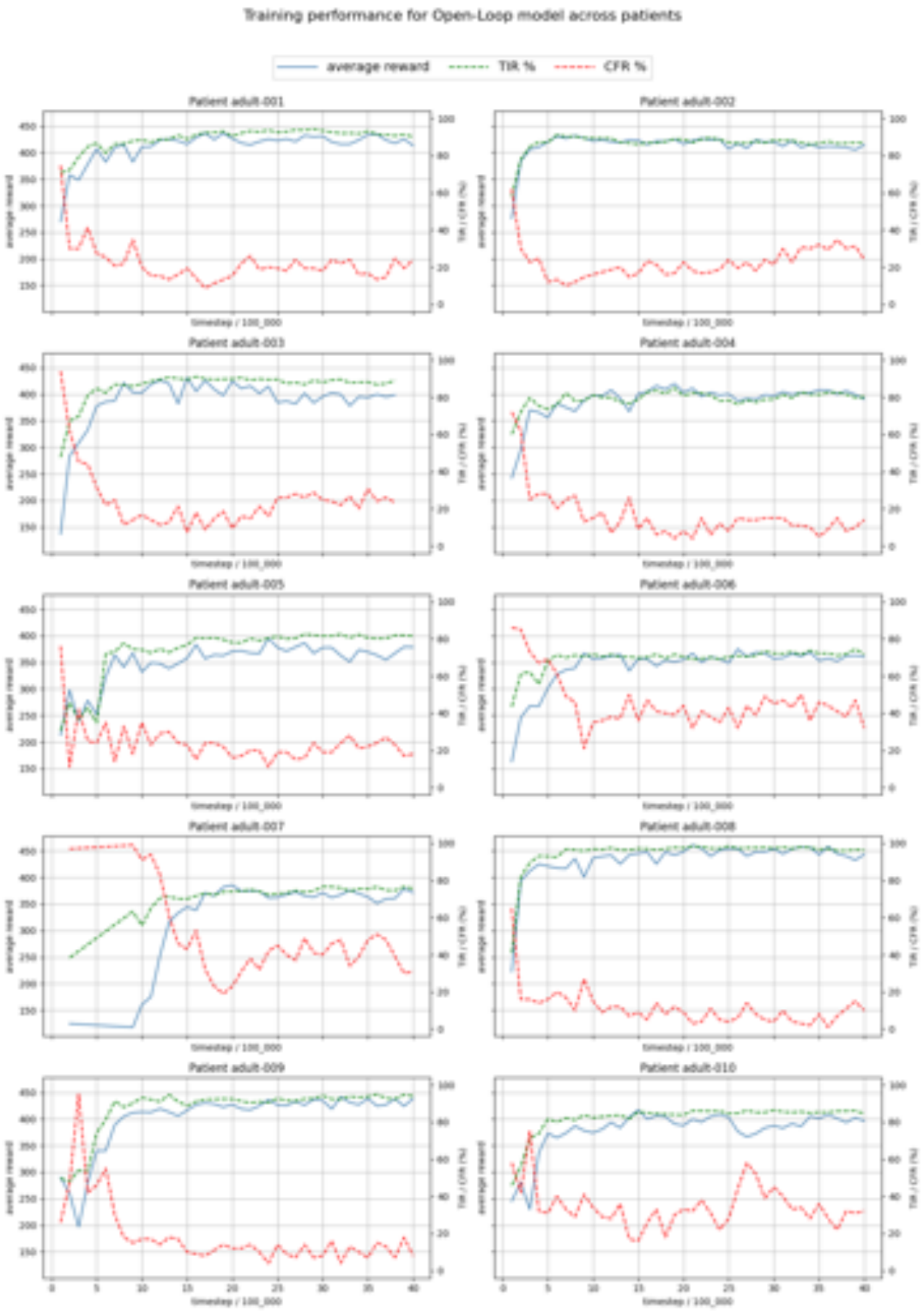


Figure 21: Overview of learning curves for individual patient-specific HCL models. Models were evaluated every 100,000 steps over 100 episodes. Blue shows average reward, green shows average TIR, and red shows CFR.

C.2 Training curves for Fully Closed-Loop model

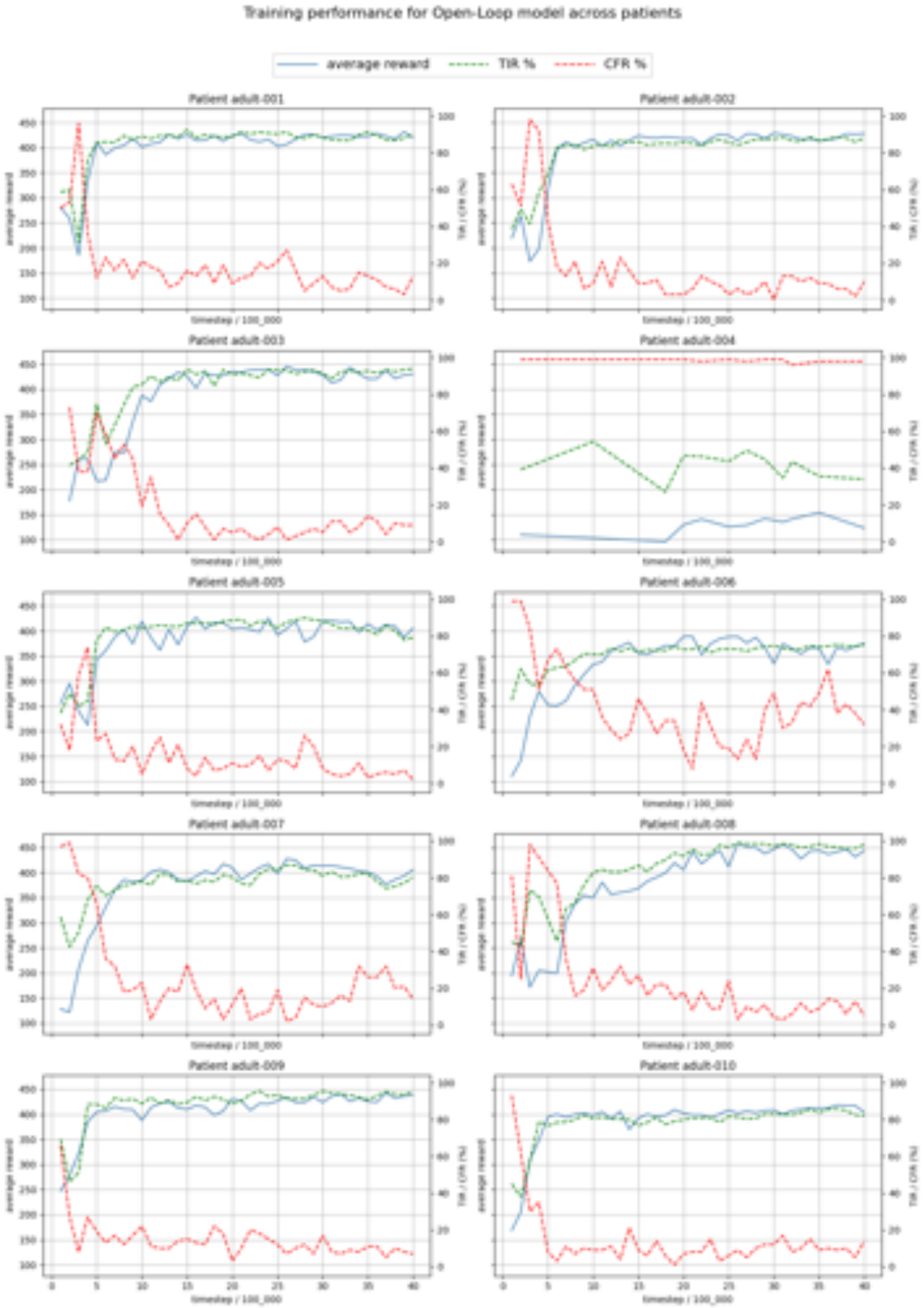


Figure 22: Overview of learning curves for individual patient-specific FCL models. Models were evaluated every 100,000 steps over 100 episodes. Blue shows average reward, green shows average TIR, and red shows CFR.